

เอกสารการใช้งาน

โปรแกรมตรวจสอบข้อมูลชีวมิติแบบผสมผสานจากอาสาสมัคร

ศูนย์เทคโนโลยีอิเล็กทรอนิกส์และคอมพิวเตอร์แห่งชาติ

เอกสารการใช้งาน

โปรแกรมตรวจสอบข้อมูลชีวมิติแบบผสมผสานจากอาสาสมัคร

จัดทำโดย

ทีมวิจัยการประมวลผลและเข้าใจภาพ (IPU)

สังกัดทีมวิจัยแกนหลักปัญญาประดิษฐ์ (AINRG)

ศูนย์เทคโนโลยีอิเล็กทรอนิกส์และคอมพิวเตอร์แห่งชาติ (NECTEC)

สงวนลิขสิทธิ์ ศูนย์เทคโนโลยีอิเล็กทรอนิกส์และคอมพิวเตอร์แห่งชาติ (NECTEC)

สารบัญ

1. คำนำ.....	4
2. ที่มาและความสำคัญของปัญหา	5
3. ข้อมูลและข้อกำหนดของไฟล์.....	6
4. โปรแกรมตรวจสอบข้อมูลชีวมิติแบบผสมผสานจากอาสาสมัคร	7
4.1 สถาปัตยกรรมระบบ	7
4.2 ไคลบรารีและโมเดลที่ใช้.....	12
5. การทำงานของโปรแกรมและตัวอย่างผลลัพธ์ที่ได้.....	13
5.1 การตรวจสอบชื่อไฟล์.....	13
5.2 การตรวจสอบข้อมูลใบหน้า	15
5.3 การตรวจสอบข้อมูลฝ่ามือ	28
5.4 การตรวจสอบข้อมูลการเดิน.....	38
6. การตั้งค่า configuration ของโปรแกรม.....	46
7. การใช้งานโปรแกรม.....	53
7.1 การใช้งานผ่าน command line.....	53
7.2 การใช้งานผ่านการเรียก API.....	58
7.3 การใช้งาน Streamlit dashboard (api_tester.py).....	59
8. Docker และ API.....	64
8.1 การติดตั้งและรัน Docker.....	64
8.2 การเรียกใช้งานผ่าน API	67
8.3 ลำดับการใช้งานทั่วไป.....	75
9. ภาคผนวก	79
9.1 การใช้งาน view_results.py dashboard	79

1. คำนำ

โครงการนี้จัดทำเพื่อตรวจสอบข้อมูลชีวิตของอาสาสมัครโดยการใช้โปรแกรมอัตโนมัติ โครงการนี้จะสำเร็จได้หากไม่มี ดร.ศรณ สุขสาตร และ ดร. ศีตภา วัชรภินชัย ผู้มีคุณูปการและอุปการะคุณช่วยเหลือให้โปรแกรมนี้สามารถสำเร็จลุล่วงออกมาได้ ทีมผู้จัดทำหวังว่าโปรแกรมนี้จะเป็นประโยชน์ต่อผู้ใช้งาน หากมีข้อผิดพลาดประการใด ทีมผู้จัดทำขออภัยมา ณ ที่นี้ด้วย

ทีมผู้จัดทำโปรแกรม

2. ที่มาและความสำคัญของปัญหา

ผู้รับจ้างต้องดำเนินการเก็บข้อมูลชีวมิติแบบผสมผสานจากอาสาสมัครไม่น้อยกว่า 1,500 คน อาสาสมัครแต่ละท่านต้องเก็บข้อมูลหลายประเภท ทีมวิจัยการประมวลผลและเข้าใจภาพ (IPU) รับผิดชอบข้อมูล 4 ประเภท ได้แก่

1. ข้อมูลภาพเคลื่อนไหวใบหน้า ใช้เครื่องถ่ายภาพความร้อนที่ทางผู้ว่าจ้างเตรียมให้
2. ข้อมูลภาพถ่ายเคลื่อนไหวจากเครื่องถ่ายภาพความร้อน ใช้เครื่องถ่ายภาพความร้อนที่ทางผู้ว่าจ้างเตรียมให้
3. ข้อมูลภาพถ่ายลายเส้นเลือดฝ่ามือ (Palm vein imaging)
4. ข้อมูลวิดีโอท่าทางการเดิน (Gait recording)

การตรวจเช็คข้อมูลของอาสาสมัครด้วยบุคคลโดยไม่อาศัยโปรแกรมอัตโนมัติเป็นเรื่องที่ทำได้ยาก เนื่องจากมีรายละเอียดข้อกำหนดของแต่ละประเภทข้อมูลที่ต้องมีการตรวจสอบเป็นจำนวนมาก หนึ่งอาสาสมัครจำเป็นต้องส่งไฟล์เป็นจำนวนมากถึง 17 ไฟล์ และจำนวนอาสาสมัครที่ต้องตรวจสอบมีปริมาณมาก จึงเป็นที่มาของโครงการสร้างระบบสำหรับช่วยในการตรวจสอบข้อมูลจากอาสาสมัคร

3. ข้อมูลและข้อกำหนดของไฟล์

อาสาสมัครหนึ่งคนต้องเก็บไฟล์ตามข้อกำหนดจำนวน 17 ไฟล์ รูปแบบของไฟล์ต่างๆ มีดังนี้

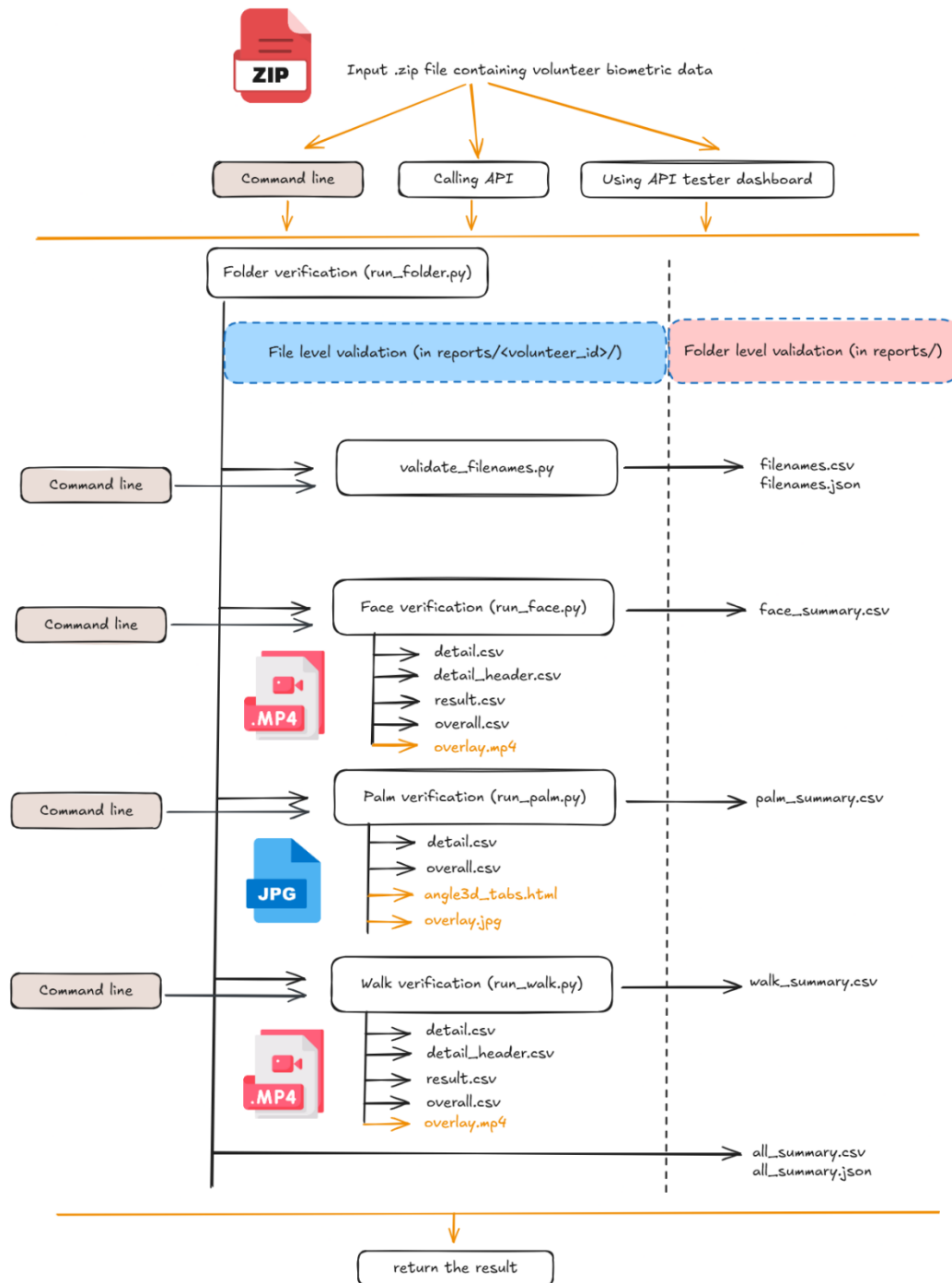
ข้อมูล	ภาพเคลื่อนไหว ใบหน้า	ภาพเคลื่อนไหวจาก เครื่องถ่ายภาพ ความร้อน	ภาพถ่ายลายเส้นเลือด ฝ่ามือ	วิดีโอท่าทาง การเดิน
ไฟล์	{รหัสแทน อาสาสมัคร) _face _rgb .mp4	(รหัสแทน อาสาสมัคร) _face _{depth, ir1, ir2, thermal} .mp4	(รหัสแทนอาสาสมัคร) _palm _{L, R} _{N, RL, RR, PU, PD} .jpg	(รหัสแทน อาสาสมัคร) _walk _{F, S} .mp4
ตัวอย่าง ไฟล์	001_face_rgb. mp4	001_face_depth. mp4	0001_palm_L_N.jpg	001_walk_F. mp4
จำนวนไฟล์ ต่อหนึ่ง อาสาสมัคร	1	4	10	2

ในแต่ละประเภทของข้อมูล มีข้อกำหนดเพิ่มเติมที่จะกล่าวถึงถัดไปในส่วนการทำงานของโปรแกรม

4. โปรแกรมตรวจสอบข้อมูลชีวมิติแบบผสมผสานจากอาสาสมัคร

4.1 สถาปัตยกรรมระบบ

ระบบตรวจสอบข้อมูลชีวมิตินี้ตรวจสอบข้อมูลของอาสาสมัคร 4 ประเภท ได้แก่ ภาพเคลื่อนไหว ใบหน้า ภาพเคลื่อนไหวจากเครื่องถ่ายภาพความร้อน ภาพถ่ายลายเส้นเลือดฝ่ามือ และ วิดีโอท่าทางการเดิน เพื่อยืนยันว่าข้อมูลดังกล่าวเป็นไปตามข้อกำหนดหรือไม่ มีสถาปัตยกรรมระบบดังนี้



การเรียกใช้งานระบบสามารถเรียกใช้งานได้จาก 3 ช่องทาง ได้แก่

1. การใช้งานผ่าน command line สามารถปรับ option ในการรันเช็คได้มากที่สุด และสามารถรันตรวจสอบแยกแต่ละส่วนของ pipeline ได้ แต่ต้องมีความเข้าใจในการรัน command และการปรับ option ต่างๆ
2. การใช้งานผ่านการเรียก API สามารถปรับ option ในการรันเช็คได้มาก แต่ต้องมีความเข้าใจในการเรียกใช้ API
3. การใช้งาน Streamlit dashboard (api_tester.py) สามารถใช้งานได้ง่ายที่สุด แต่สามารถปรับแต่ง option ในการรันเช็คได้น้อยที่สุด

เมื่อรับข้อมูลนำเข้าในรูปแบบของ zip file ที่ประกอบไปด้วยข้อมูลชีวมิติของอาสาสมัครต่าง จะทำการรัน run_folder.py ซึ่งระบบจะสแกนด้วยความลึก 5 ระดับชั้นของ folder เพื่อหาไฟล์ของอาสาสมัครแต่ละราย การเช็คจะทำได้ 2 ระดับ ได้แก่ การเช็คระดับ file level validation เพื่อเช็คแต่ละไฟล์ตรงตามข้อกำหนดของประเภทชีวมิติของไฟล์นั้นหรือไม่ และ การเช็คระดับ folder level validation เพื่อเช็คผลลัพธ์รวมทั้ง folder

การตรวจตามลำดับขั้นตอน pipeline มีดังนี้

- **validate_filenames.py** เช็คชื่อไฟล์แต่ละไฟล์ว่าตรงตามข้อกำหนดหรือไม่ และอาสาสมัครแต่ละรายส่งไฟล์มาครบจำนวน 17 ไฟล์หรือไม่ หากไฟล์ใดไป จะรายงานผลลัพธ์การเช็คออกมาทาง filenames.csv และ filenames.json และทำการดึงค่า volunteer id, ประเภทของชีวมิติ (face, palm, walk) เพื่อดำเนินการใน pipeline ต่อไป
- **run_face.py** ตรวจสอบข้อมูลชีวมิติประเภทภาพเคลื่อนไหวใบหน้า รายงานผลในระดับ folder level check ออกมาทาง face_summary.csv
- **run_palm.py** ตรวจสอบข้อมูลชีวมิติประเภทภาพถ่ายลายเส้นเลือดฝ่ามือ รายงานผลในระดับ folder level check ออกมาทาง palm_summary.csv
- **run_walk.py** ตรวจสอบข้อมูลชีวมิติประเภทวิดีโอท่าทางการเดิน รายงานผลในระดับ folder level check ออกมาทาง walk_summary.csv
- **run_folder.py** เขียนผลลัพธ์ภาพรวมทั้งหมดที่ได้ลงใน all_summary.csv

หลังจากนั้นจึงส่งผลลัพธ์คืนตามช่องทางการใช้งานระบบ โดยภาพเคลื่อนไหวจากเครื่องถ่ายภาพความร้อน จะผ่านการเช็คเพียงแค่ขั้นตอน **validate_filenames.py** เท่านั้น เพื่อเช็คที่อาสาสมัครมีการส่งข้อมูลมาจริง

แต่จะไม่มีการใช้รายละเอียดของไฟล์ และนอกจากนี้ในชีวมิติแต่ละประเภทจะมีการรายงานผลจากการเช็คในระดับ file level validation แยกสำหรับแต่ละประเภทของไฟล์ในโฟลเดอร์ของ volunteer_id นั้นๆ ซึ่งจะกล่าวในรายละเอียดการเช็คในแต่ละประเภทของข้อมูลชีวมิติในส่วนถัดไป

ผลลัพธ์ของการรันโปรแกรมผ่านไฟล์ `run_folder.py` มีดังนี้

- ผลลัพธ์แยกของแต่ละอาสาสมัครจะอยู่ใน `reports/<volunteer_id>` และมี video overlay หรือ image overlay สำหรับการ debug ในแต่ละประเภทของข้อมูลชีวมิติ ในกรณีภาพฝ่ามือจะมี `_angle3d_tabs.html` เพื่อให้ตรวจสอบและระนาบของฝ่ามือที่โปรแกรมอ่านได้ผ่านภาพสามมิติสำหรับไฟล์ชีวมิติหนึ่งไฟล์อาจมีรายงานผลดังนี้
 - ไฟล์ `detail.csv` บันทึกผลการ check อย่างละเอียดในแต่ละเฟรม จะตรวจสอบใน 3 ระดับ ได้แก่ video, frame, sequence พร้อมบอกสถานะว่าแต่ละเฟรม FAIL, PASS, หรือ SKIP พร้อมทั้งเหตุผล
 - ไฟล์ `detail_header.csv` บันทึกค่าเพิ่มเติมที่วัดได้ในแต่ละเฟรม เช่น เวลา มุมศีรษะ (yaw, pitch, roll) ความสว่าง ความคมชัด ค่าการตรวจจับสิ่งกีดขวางในแต่ละจุด landmark ขนาดของบุคคลที่วัดได้ในเฟรม
 - ไฟล์ `result.csv` สรุปผลแยกแต่ละ check แบบรวมทุกเฟรม
 - ไฟล์ `overall.csv` รายงานผลลัพธ์สุดท้ายของไฟล์ชีวมิติหนึ่งไฟล์ว่าเป็น PASS หรือ FAIL พร้อมบอก check ที่ไม่ผ่าน
 - ไฟล์ `overlay.mp4` และ `overlay.jpg` แสดง landmark, bounding box และผลการตรวจสอบแต่ละ check บนภาพหรือวิดีโอ
 - ไฟล์ `angle3d_tabs.html` ใช้แสดงค่ามุมของฝ่ามือในรูปแบบสามมิติสำหรับช่วยตรวจสอบและแก้ไขข้อผิดพลาดของโค้ด
- ผลลัพธ์รวมจะอยู่ใน `reports/` ได้แก่
 - ไฟล์ `face_summary.csv`, `palm_summary.csv`, `walk_summary.csv` ใช้สรุปสถานะของไฟล์ทั้งหมดในชีวมิติประเภทเดียวกัน
 - ไฟล์ `all_summary.csv` / `all_summary.json` ผลลัพธ์สรุปรวมของทุกข้อมูล ทั้งใบหน้า ฝ่ามือ และการเดิน
 - ไฟล์ `filenames.csv` / `filenames.json` เป็นผลลัพธ์ที่ได้จาก `validate_filenames.py` ใช้รายงานว่าอาสาสมัครแต่ละรายส่งไฟล์ครบตามข้อกำหนดจำนวน 17 ไฟล์หรือไม่ ไฟล์ที่มี

ปัญหา เช่น ชื่อไฟล์ไม่ตรงตามข้อกำหนดหรือ RegEx จะถูกข้าม และไม่นำไปตรวจสอบ

ต่อไป

ตัวอย่างข้อมูลนำเข้าที่อยู่ใน folder data/ ในกรณีที่ volunteer 001 ส่งมาเพียงไฟล์ประเภทชีวมิติใบหน้า แต่ volunteer 002 ส่งไฟล์มาครบและถูกต้องทุกชีวมิติ

data

```
|-----001
|
|   001_face_depth.mp4
|
|   001_face_ir1.mp4
|
|   001_face_ir2.mp4
|
|   001_face_rgb.mp4
|
|   001_face_thermal.mp4
|
|-----002
|
|   002_face_depth.mp4
|
|   002_face_ir1.mp4
|
|   002_face_ir2.mp4
|
|   002_face_rgb.mp4
|
|   002_face_thermal.mp4
|
|   002_palm_L_N.jpg
|
|   002_palm_L_PD.jpg
|
|   002_palm_L_PU.jpg
|
|   002_palm_L_RL.jpg
|
|   002_palm_L_RR.jpg
|
|   002_palm_R_N.jpg
|
|   002_palm_R_PD.jpg
|
|   002_palm_R_PU.jpg
|
|   002_palm_R_RL.jpg
```

002_palm_R_RR.jpg

002_walk_F.mp4

002_walk_S.mp4

ผลลัพธ์ของการรัน **run_folder.py** หนึ่งครั้งจะมีลักษณะดังนี้ (ดูรายละเอียดเพิ่มเติมได้ในหัวข้อ 7.1 การใช้
งานโปรแกรมผ่าน command line) โดยอยู่ใน reports/

reports

| all_summary.csv

| all_summary.json

| filenames.csv

| filenames.json

| face_summary.csv

| palm_summary.csv

| walk_summary.csv

|

|-----001

| face_001_detail.csv

| face_001_detail_header.csv

| face_001_overall.csv

| face_001_result.csv

| face_001_overlay.mp4

|

|-----002

face_002_{detail, detail_header, overall, result}.csv **x 4 files**

face_002_overlay.mp4

palm_002_angle3d_tabs.html

palm_002_{detail, overall}.csv **x 2 files**

palm_002_angle3d.html

palm_002_{L, R}_{N, PD, PU, RL, RR}_overlay.jpg × 10 files

walk_002_{F, S}_{detail, detail_header, overall, result}.csv × 8 files

walk_002_F_overlay.mp4

walk_002_S_overlay.mp4

4.2 ไลบรารีและโมเดลที่ใช้

โมเดลหลักที่ใช้ในการตรวจสอบคือ pretrained model ในตระกูล MediaPipe ดังนี้

- การตรวจสอบ face ใช้ MediaPipe Face Landmarker
- การตรวจสอบ palm ใช้ MediaPipe Hand Landmarker
- การตรวจสอบ walk ใช้ MediaPipe Pose Landmarker สำหรับการตรวจจับบุคคล และใช้ YOLOv8-nano สำหรับการตรวจสอบสิ่งกีดขวาง

และมีไลบรารีอื่นๆ ที่ใช้ เช่น

- สำหรับการตรวจสอบใน pipeline มีการใช้ Numpy, OpenCV, และ Protobuf
- สำหรับการทำ API มีการใช้ FastAPI, Python multipart, และ Uvicorn
- สำหรับการทำ dashboard และ visualization มีการใช้ Streamlit, Altair, Pandas, และ Plotly

5. การทำงานของโปรแกรมและตัวอย่างผลลัพธ์ที่ได้

5.1 การตรวจสอบชื่อไฟล์

ไฟล์ : qc/validate_filenames.py

ในแต่ละไฟล์จะตรวจสอบว่าชื่อไฟล์เป็นไปตามข้อกำหนดหรือไม่ โดยการเทียบกับ RegEx ใน config.yml ในส่วนของ filenames: required: ซึ่งมีทั้งหมด 17 RegEx ในการตรวจสอบ ผลลัพธ์ที่ได้ใน filenames.csv คือช่อง status รายงานผลว่าอาสาสมัครส่งไฟล์ครบหรือไม่

- หากส่งไฟล์ครบ จะรายงานเป็น 1 แถวต่อหนึ่งอาสาสมัคร ระบุว่า volunteer หมายเลขอะไร และ status COMPLETE
- หากส่งไฟล์ไม่ครบ จะรายงานเป็น 1 แถวต่อหนึ่งไฟล์ที่มีปัญหา ในแต่ละแถวรายงานหมายเลข volunteer และ status INCOMPLETE
- หากไม่พบชื่อไฟล์ตรงกับ RegEx ใดเลยจะรายงานว่ามีไฟล์นั้น status UNRECOGNISED

ช่อง issue type รายงานปัญหาแยกในแต่ละไฟล์ (1 แถวต่อหนึ่งไฟล์ที่มีปัญหา)

- หากอาสาสมัครส่งไฟล์ไม่ครบตามจำนวน 17 ไฟล์ จะรายงานชื่อไฟล์ที่ขาดของอาสาสมัครนั้น และมี issue_type MISSING
- หากพบชื่อไฟล์ว่าตรงกับ RegEX แต่อาสาสมัครส่งไฟล์มากเกินไปเกินจำนวนที่กำหนด เช่น มีไฟล์ data/002/002_face_rgb.mp4 และ data/002_face_rgb.mp4 ในกรณีนี้จะถือว่าอาสาสมัครส่งไฟล์ซ้ำมิติใบหน้าประเภทภาพสี RGB มา 2 ไฟล์เพียงแต่อยู่ในคนละ folder ซึ่งถือว่าเกินจำนวนที่กำหนด คือ 1 ไฟล์ จะแสดง issue_type DUPLICATE
- หากไม่พบชื่อไฟล์ตรงกับ RegEX ใดเลยจะรายงานว่ามีไฟล์นั้น issue_type UNRECOGNISED

ตัวอย่างผลการรันแสดง filenames.csv โดยใช้ extension Data Wrangler ใน Visual Studio Code

# volunteer	status	issue_type	item	path
Missing: 0 (0%) Distinct: 4 (57%)	Missing: 0 (0%) Distinct: 3 (43%)	Missing: 0 (0%) Distinct: 3 (43%)	Missing: 1 (14%) Distinct: 3 (43%)	Missing: 2 (29%) Distinct: 3 (43%)
	INCOMPLETE COMPLETE UNRECOGNISED	71% missing 14% duplicate 14% unrecognised	43% face_rgb 29% palm_R_PD 14% walk_S	43% .\data\mock\mock_filename... 14% .\data\mock\mock_filename... 14% .\data\mock\mock_filename...
0	1	COMPLETE	Missing value	Missing value
1	2	INCOMPLETE	missing	palm_R_PD
2	2	INCOMPLETE	missing	walk_S
3	3	INCOMPLETE	missing	face_rgb
4	4	INCOMPLETE	duplicate	face_rgb
5	4	INCOMPLETE	duplicate	face_rgb
6	3	UNRECOGNISED	unrecognised	Missing value

5.2 การตรวจสอบข้อมูลใบหน้า

ไฟล์ : qc/pipelines/face_rgb.py

สั่งรันตรวจสอบข้อมูลชีวมิติประเภทใบหน้า

ไฟล์ : run_face.py

ประกอบไปด้วยการเช็คหลากหลายระดับ สถานะผลลัพธ์การเช็คที่มีได้คือ PASS SKIP FAIL โดย PASS

แปลว่าการเช็คนั้นผ่าน SKIP ความหมายคือจะไม่ทำการเช็คนั้นในเฟรมนั้นๆ FAIL แปลว่าการเช็คนั้นไม่ผ่าน

โดยในแต่ละวิดีโอจะต้องไม่มีสถานะ FAIL แม้แต่การเช็คเดียว มิฉะนั้นจะถือว่าทั้งวิดีโอไม่ผ่านตามข้อกำหนด ซึ่งมีการเช็คต่างๆ มีดังนี้

5.2.1 การเช็คระดับ video

เป็นการเช็คข้อมูลภาพรวมทั้งวิดีโอ ก่อนเข้าสู่ขั้นตอนการเช็คแต่ละเฟรมของวิดีโอ

check_container, check_fps, check_duration, check_resolution

เป็นการเช็คระดับ video สอดคล้องกับความต้องการระบบที่ว่า

“ภาพที่มีคุณสมบัติเป็นภาพสี (RGB) และไฟล์บันทึกเป็น MPEG-4 (*.mp4)”

“ในภาพวิดีโอ ทำต่อเนื่องจนครบโดยใช้เวลา อย่างน้อย 40 วินาทีต่อวิดีโอต่อคน”

“ภาพควรมีความละเอียดไม่น้อยกว่า 5 เฟรมต่อวินาที”

“รูปแบบการบันทึกชื่อไฟล์ 001_face_rgb.mp4 โดย 001 หมายถึง รหัสแทนอาสาสมัคร, face หมายถึง ชีวมิติใบหน้า, .mp4 หมายถึง ชนิดของการจัดเก็บข้อมูล”

การทำงานโดยหลักมีดังนี้

ไฟล์ `utils/video.py` ฟังก์ชัน `probe_video`

```
def probe_video(path: PathLike, *, decode_first_frame: bool = True) -> VideoMetadata:
    cv2 = _require_cv2()
    p = _normalise_path(path)

    if not p.exists():
        return VideoMetadata(
            path=str(p),
            filename=p.name,
            extension=p.suffix.lower(),
            readable=False,
            width=None,
            height=None,
```

```

        fps=None,
        frame_count=None,
        duration_sec=None,
        codec_fourcc=None,
        channel_count=None,
        reason="file does not exist",
    )

    if not p.is_file():
        return VideoMetadata(
            path=str(p),
            filename=p.name,
            extension=p.suffix.lower(),
            readable=False,
            width=None,
            height=None,
            fps=None,
            frame_count=None,
            duration_sec=None,
            codec_fourcc=None,
            channel_count=None,
            reason="path is not a file",
        )

    cap = cv2.VideoCapture(str(p))
    if not cap.isOpened():
        cap.release()
        return VideoMetadata(
            path=str(p),
            filename=p.name,
            extension=p.suffix.lower(),
            readable=False,
            width=None,
            height=None,
            fps=None,
            frame_count=None,
            duration_sec=None,
            codec_fourcc=None,
            channel_count=None,
            reason="OpenCV could not open video",
        )

    fps = _clean_float(cap.get(cv2.CAP_PROP_FPS))
    frame_count = _clean_int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    width = _clean_int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = _clean_int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    codec_fourcc = _fourcc_to_string(cap.get(cv2.CAP_PROP_FOURCC))
    channel_count: Optional[int] = None
    is_color: Optional[bool] = None
    readable = True
    reason = ""

    if decode_first_frame:
        ok, frame = cap.read()
        if not ok or frame is None:
            readable = False
            reason = "OpenCV opened file but could not decode first frame"
        else:
            frame_h, frame_w = frame.shape[:2]
            width = width or int(frame_w)
            height = height or int(frame_h)
            channel_count = _frame_channel_count(frame)
            is_color = _is_truly_RGB(frame)

    cap.release()

```



```

duration_sec: Optional[float]
if fps is not None and frame_count is not None:
    duration_sec = frame_count / fps
else:
    duration_sec = None

return VideoMetadata(
    path=str(p),
    filename=p.name,
    extension=p.suffix.lower(),
    readable=readable,
    width=width,
    height=height,
    fps=fps,
    frame_count=frame_count,
    duration_sec=duration_sec,
    codec_fourcc=codec_fourcc,
    channel_count=channel_count,
    is_color=is_color,
    reason=reason,
)

```

ทำหน้าที่อ่านข้อมูล metadata ของไฟล์วิดีโอ โดยการตรวจสอบไฟล์ ตรวจสอบ path และให้ OpenCV ทดลองเปิดไฟล์ และทดลองถอดเฟรมแรกของไฟล์ คืนค่ากลับเป็น instance type VideoMetadata ที่มีค่า metadata ต่างๆ ซึ่งจะนำไปใช้ต่อไปในการตรวจสอบวิดีโอทุกประเภท ทั้งชีวิตมิติใบหน้าและท่าทางการเดิน โดยผลลัพธ์ที่ได้จะถูกส่งต่อไปยังการเช็คในลำดับถัดไป

ไฟล์ qc/check_metadata.py ฟังก์ชัน check_container

```

def check_container(meta, *, require_rgb: bool = True, expected_ext: str = ".mp4"):
    """Is the file a readable .mp4, and (for rgb) 3-channel color?"""
    if not meta.readable:
        return (FAIL, f"unreadable: {meta.reason}")
    if meta.extension != expected_ext:
        return (FAIL, f"extension={meta.extension} != {expected_ext}")
    if require_rgb:
        if meta.channel_count is None or meta.channel_count < 3:
            return (FAIL, "channel count unknown (cannot verify RGB)")
        if not meta.is_color:
            return (FAIL, "grayscale content (channels identical; not true RGB)")
    return (PASS, f"container ok ({meta.extension}, "
        f"{meta.channel_count} channels, codec={meta.codec_fourcc})")

```

มีการเช็คไฟล์ว่าอ่านได้ และ เป็นนามสกุล .mp4 และ content มีสี RGB จึงจะคืนค่ากลับเป็น TRUE มิฉะนั้นจะคืนค่ากลับเป็น FALSE พร้อมเหตุผลประกอบ

ไฟล์ qc/check_metadata.py ฟังก์ชัน check_fps

```

def check_fps(meta, *, min_fps: float = 5.0):
    if meta.fps is None:
        return (FAIL, "fps unknown (cannot verify against spec)")
    if meta.fps >= min_fps:
        return (PASS, f"fps={meta.fps:.2f} >= {min_fps}")
    return (FAIL, f"fps={meta.fps:.2f} < {min_fps}")

```

เช็คค่า fps ว่ามากกว่า 5.0 จึงจะคืนค่ากลับเป็น TRUE มิฉะนั้นจะคืนค่ากลับเป็น FALSE พร้อมเหตุผลประกอบ

ไฟล์ `qc/check_metadata.py` ฟังก์ชัน `check_duration`

```
def check_duration(meta, *, min_duration_sec: float = 40.0):
    if meta.duration_sec is None:
        return (FAIL, "duration unknown (cannot verify against spec)")
    if meta.duration_sec >= min_duration_sec:
        return (PASS, f"duration={meta.duration_sec:.1f} >= {min_duration_sec}")
    return (FAIL, f"duration={meta.duration_sec:.1f} < {min_duration_sec}")
```

เช็คค่า duration ของวิดีโอว่ามากกว่า 40.0 วินาที จึงจะคืนค่ากลับเป็น TRUE มิฉะนั้นจะคืนค่ากลับเป็น FALSE พร้อมเหตุผลประกอบ

ไฟล์ `qc/check_metadata.py` ฟังก์ชัน `check_resolution`

```
def check_resolution(meta, *, min_width: int = 180, min_height: int = 180):
    if meta.width is None or meta.height is None:
        return (FAIL, "resolution unknown (cannot verify against spec)")
    if meta.width >= min_width and meta.height >= min_height:
        return (PASS, f"resolution={meta.width}x{meta.height} "
                    f">= {min_width}x{min_height}")
    return (FAIL, f"resolution={meta.width}x{meta.height} "
                f"< {min_width}x{min_height}")
```

เช็คค่า resolution รวมทั้งหมดของวิดีโอว่ามากกว่า 180 × 180 จึงจะคืนค่ากลับเป็น TRUE มิฉะนั้นจะคืนค่ากลับเป็น FALSE พร้อมเหตุผลประกอบ

5.2.2 การเช็คระดับ frame

ทำงานกับทุกเฟรมที่สุ่มมาจากวิดีโอตัวอย่าง และสรุปผลรวมด้วยสัดส่วนเฟรมที่ผ่าน เทียบว่าหากสัดส่วนเฟรมที่ผ่านมีมากกว่าเกณฑ์ที่ตั้งไว้ในแต่ละประเภทการเช็คจะถือว่าการเช็คในหัวข้อนั้นของวิดีโอมีสถานะ PASS แต่หากมีเฟรมที่ผ่านน้อยกว่าเกณฑ์จะถือว่าการเช็คในหัวข้อนั้นวิดีโอมีสถานะ FAIL โดยสามารถตั้งค่าเกณฑ์ได้ในไฟล์ `config.yml` (ดูรายละเอียดเพิ่มเติมได้ในส่วน 6. การตั้งค่า configuration ของโปรแกรม)

check_face_detected

ไฟล์ `qc/pipelines/face_rgb.py` ฟังก์ชัน `run_face_rgb` ซึ่งมีการเรียกใช้ฟังก์ชัน

`create_face_landmarker`, `detect_face` จาก `qc/checks/face_landmarker`

```
fr = detect_face(
    img, detector=face_landmarker, input_color_space=cspace)
ok = fr.ok
msg = fr.message
landmarks = fr.landmarks_px          # pixel-space, get_lm-compatible
```

```

bbox = fr.bbox
blendshapes = fr.blendshapes
norm_landmarks = fr.landmarks_norm # raw normalized, for head pose
if not ok:
    no_faces = msg.startswith("No faces")
    multiple_faces = msg.startswith("Multiple faces detected")

    if no_faces:
        add("check_face_detected", "SKIP",
            f"frame={sf.frame_index} {msg}", sf.frame_index)
        gap_row_positions[len(timeline)] = len(rows) - 1
    else:
        add("check_face_detected", "FAIL",
            f"frame={sf.frame_index} {msg}", sf.frame_index)

add_frame(sf, face_detected=False)
if overlay is not None:
    overlay.add_frame(
        img, cspace, sf.frame_index, sf.timestamp_sec,
        face_detected=False, landmarks=None, bbox=None,
        pose=None, label="no-face", checks=dict(_frame_checks))

if multiple_faces:
    sequence_blocked_by_structural_fail = True

if fail_fast:
    aborted = True
    break
continue
add("check_face_detected", "PASS",
    f"frame={sf.frame_index} face detected", sf.frame_index)

```

ฟังก์ชัน detect_face จะทำการตรวจจับใบหน้าจากการเรียกใช้โมเดล MediaPipe Face Landmarker เพื่อตรวจสอบแต่ละเฟรม ผ่านการใช้ Tasks API และเปิดให้แสดงผล output_face_blendshapes=True ควบคู่ไปกับการแสดง landmarks ของใบหน้าที่ตรวจจับได้ และโปรแกรมจะเช็คผลที่ได้รับกลับมาผ่านทาง instance type FaceResult

```

@dataclass
class FaceResult:
    ok: bool
    message: str
    landmarks_px: Optional[list] = None # [(x, y, z)] pixel space
    landmarks_norm: Optional[Any] = None # raw normalized landmark list (.x/.y/.z)
    bbox: Optional[tuple] = None # (x, y, w, h) px, 10% margin
    norm_box: Optional[tuple] = None # (x, y, w, h) normalized
    blendshapes: Optional[dict] = None # {name: score}

```

หลังจากนั้นจะแปลผลที่ได้จาก instance type FaceResult ต่างๆ เช่น ตรวจจับใบหน้าพบเพียง 1 ใบหน้าเท่านั้น ไม่มีการตรวจจับพบหลายใบหน้า การเช็คนี้เป็นการเช็คลำดับแรก ซึ่งการเช็คระดับ frame รายการอื่นต้องอาศัยผลลัพธ์ PASS ของการเช็คนี้ หากตรวจไม่พบใบหน้าในเฟรมใด การเช็คอื่นๆ ในเฟรมนั้นจะไม่สามารถทำงานได้ ผลลัพธ์การเช็คอื่นๆ ในระดับเฟรมจะถูกบันทึก เป็น SKIP หรือแปลว่าจะข้ามการเช็คนั้นไป แต่หากไม่ผ่านเงื่อนไข เช่น พบมากกว่า 1 ใบหน้าในเฟรม จะทำให้เฟรมนั้นแปลผลเป็น FAIL (ส่งคืนค่าใน FaceResult กลับเป็น ok: False)

check_head_fully

ไฟล์ `qc/checks/check_head_fully.py`

```
_TOP_OF_HEAD_IDX = 10
_CHIN_IDX = 152
_LEFT_CHEEK_IDX = 127
_RIGHT_CHEEK_IDX = 356

def check_head_fully(landmarks, image_height: int, image_width: int, margin_px: int = 10):
    if not landmarks:
        return (False, "No landmarks provided")

    try:
        top_y = landmarks[_TOP_OF_HEAD_IDX][1]
        chin_y = landmarks[_CHIN_IDX][1]
        left_cheek_x = landmarks[_LEFT_CHEEK_IDX][0]
        right_cheek_x = landmarks[_RIGHT_CHEEK_IDX][0]
    except (IndexError, TypeError):
        return (False, "Landmark list missing required points")

    top_cut = top_y < margin_px
    chin_cut = chin_y > image_height - margin_px
    left_cut = left_cheek_x < margin_px
    right_cut = right_cheek_x > image_width - margin_px

    if top_cut and chin_cut:
        return (False, "Top of head and chin might be cut")
    if top_cut:
        return (False, "Top of head might be cut")
    if chin_cut:
        return (False, "Chin might be cut")
    if left_cut:
        return (False, "Left cheek might be cut")
    if right_cut:
        return (False, "Right cheek might be cut")
    return (True, "Head is fully visible")
```

สอดคล้องกับความต้องการระบบ “ใบหน้าต้องชัดเจน สว่างเพียงพอให้เห็นรายละเอียดอวัยวะบนใบหน้าชัดเจน และศีรษะตั้งตรง” ในแต่ละเฟรมเชื่อว่าใบหน้าอยู่ในกรอบเฟรมครบถ้วนหรือไม่ โดยใช้ landmark 4 จุดที่เป็นขอบเขตของศีรษะ ได้แก่ กลางหน้าผาก (จุดที่ 10) ปลาญคาง (จุดที่ 152) แก้มซ้าย (จุดที่ 127) และแก้มขวา (จุดที่ 356) หากจุดใดอยู่ใกล้ขอบภาพเกินกว่าค่า `margin_px` (ซึ่งมีค่า default ที่ 10 พิกเซล) จะถือว่าส่วนนั้นถูกตัดออกจากเฟรม และคืนค่ากลับเป็น FALSE พร้อมระบุเหตุผลว่าส่วนใดถูกตัด แต่หากพบทั้ง 4 landmarks จะคืนค่ากลับเป็น TRUE

check_face_size

ไฟล์ `qc/checks/check_face_size.py`

```
def check_face_min_size(bbox, min_width: int = 180, min_height: int = 180):
    if bbox is None:
        return (False, "No bounding box provided")

    x, y, w, h = bbox
```

```

if w >= min_width and h >= min_height:
    return (True, f"face={w}x{h} >= {min_width}x{min_height}")
return (False, f"face={w}x{h} < {min_width}x{min_height}")

```

สอดคล้องกับส่วน “ขนาดกว้างและยาวบริเวณศีรษะไม่น้อยกว่า 180 x 180 pixel” เช็ขนาด bounding box (bbox) ที่ตรวจจับและคำนวณได้ โดยคำนวณ bbox จากขนาดความกว้างยาวของ landmarks ที่ตรวจจับได้ แล้วบวก margin 10% สอดคล้องกับความต้องการระบบที่ว่า "ขนาดกว้างและยาวบริเวณศีรษะไม่น้อยกว่า 180 x 180 pixel"

check_eyes_open

ไฟล์ qc/checks/check_eye.py

```

def check_eye_status(blendshapes: Optional[dict],
                    blink_threshold: float = 0.5,
                    *,
                    return_scores: bool = False):
    def _result(ok, msg, left=None, right=None):
        if return_scores:
            return (ok, msg, left, right)
        return (ok, msg)

    if not blendshapes:
        return _result(False, "No blendshapes provided")

    left = blendshapes.get(BLINK_LEFT)
    right = blendshapes.get(BLINK_RIGHT)

    if left is None or right is None:
        return _result(False, "Blink blendshapes missing (eyeBlinkLeft/Right)")

    # Low blink score = open. Both eyes must be open to pass.
    if left < blink_threshold and right < blink_threshold:
        return _result(True,
                        f"eyes open (blinkL={left:.2f}, blinkR={right:.2f} <
{blink_threshold})",
                        left, right)
    return _result(False,
                    f"eye(s) closed (blinkL={left:.2f}, blinkR={right:.2f} >=
{blink_threshold})",
                    left, right)

```

เชื่อว่าอาสาสมัครลืมตาทั้งสองข้างหรือไม่ โดยใช้ค่า blendshape ที่ MediaPipe Face Landmarker คำนวณให้ ได้แก่ eyeBlinkLeft และ eyeBlinkRight ซึ่งเป็นคะแนนในช่วง 0 ถึง 1 โดยค่าที่สูงเข้าใกล้ 1 หมายถึงตาปิด ค่าที่ต่ำเข้าใกล้ 0 หมายถึงตาเปิด

ดวงตาทั้งสองข้างต้องมีคะแนนต่ำกว่าเกณฑ์ blink_threshold (กำหนดไว้ 0.5 ตาม config.yml) จึงจะคืนค่ากลับเป็น TRUE มิฉะนั้นจะคืนค่ากลับเป็น FALSE พร้อมระบุคะแนนที่วัดได้ทั้งสองข้าง

check_brightness

ไฟล์ qc/checks/check_brightness.py

```
def check_brightness(
    image: Any,
    bbox: Tuple[int, int, int, int],
    dark_threshold: float = 35.0,
    bright_threshold: float = 200.0,
    margin: float = 0.1,
    *,
    input_color_space: Literal["BGR", "RGB"] = "BGR",
) -> Tuple[bool, str]:
    img = _to_bgr(image, input_color_space)
    if img is None or getattr(img, "size", 0) == 0:
        return (False, "invalid_image")

    h_img, w_img = img.shape[:2]
    x, y, bw, bh = bbox

    # Central region: trim `margin` off each side of the supplied bbox
    x_start = max(0, int(x + bw * margin))
    y_start = max(0, int(y + bh * margin))
    x_end = min(w_img, int(x + bw * (1 - margin)))
    y_end = min(h_img, int(y + bh * (1 - margin)))
    if x_end <= x_start or y_end <= y_start:
        return (False, "invalid_crop")

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    region_v = hsv[y_start:y_end, x_start:x_end, 2]
    if region_v.size == 0:
        return (False, "empty_region")

    brightness = float(np.mean(region_v))

    if brightness < dark_threshold:
        status = "too_dark"
    elif brightness > bright_threshold:
        status = "too_bright"
    else:
        status = "normal"

    b = int(round(brightness))
    lo = int(round(dark_threshold))
    hi = int(round(bright_threshold))

    if status == "normal":
        msg = f"brightness={b}; normal; {lo} <= {b} <= {hi}"
    elif status == "too_dark":
        msg = f"brightness={b} < {lo}; too_dark"
    else: # too_bright
        msg = f"brightness={b} > {hi}; too_bright"

    return (status == "normal", msg)
```

สอดคล้องกับความต้องการระบบ “ใบหน้าต้องชัดเจน สว่างเพียงพอให้เห็นรายละเอียดอวัยวะบนใบหน้า

ชัดเจน และศีรษะตั้งตรง” เช็คว่าความสว่างของภาพบริเวณใบหน้า โดยแปลงภาพเป็น HSV แล้ววัดค่าเฉลี่ยของ channel V (Value) แทนความสว่าง โดยตัดขอบของกรอบใบหน้าออกตามค่า margin หากค่าที่วัดได้อยู่

ในช่วง 35 ถึง 220 จะถือว่าเป็นแสงปกติและคืนค่ากลับเป็น TRUE มิฉะนั้นจะคืนค่ากลับเป็น FALSE พร้อมระบุว่ามืดเกินไป (too_dark) หรือสว่างเกินไป (too_bright)

check_face_blur

ไฟล์ qc/checks/check_face_blur.py

```
ycrcb = cv2.cvtColor(face, cv2.COLOR_BGR2YCrCb)
y = ycrcb[:, :, 0]

mean_luma = float(np.mean(y))
contrast = float(np.std(y))

if mean_luma < min_luma:
    return (
        None,
        f"blur not judged: face too dark; "
        f"luma={mean_luma:.1f} < {min_luma:.1f}",
    )

if contrast < min_contrast:
    return (
        None,
        f"blur not judged: too little local contrast; "
        f"contrast={contrast:.1f} < {min_contrast:.1f}",
    )
```

สอดคล้องกับความต้องการระบบ “ใบหน้าต้องชัดเจน สว่างเพียงพอให้เห็นรายละเอียดของใบหน้า ชัดเจน และศีรษะตั้งตรง” เช็คว่าความคมชัดของภาพบริเวณใบหน้า โดยแปลงภาพเป็นปริภูมิสี YCrCb แล้วใช้เฉพาะ channel Y (Luminance) ซึ่งเป็นช่องสัญญาณที่บรรจุรายละเอียดของขอบภาพ จากนั้นปรับคอนทราสต์ด้วยวิธี CLAHE และให้คะแนนความคมชัดด้วยวิธี Tenengrad ซึ่งคำนวณจากขนาดของ gradient ที่ได้จาก Sobel operator ภาพที่คมชัดจะมีขอบที่ชัดเจนจึงได้คะแนนสูง ส่วนภาพที่เบลอจะได้คะแนนต่ำ แล้วนำไปแปลผลค่าความคมชัดเทียบกับ threshold เพื่อประเมินว่า PASS ถ้ามากกว่าหรือเท่ากับ threshold, FAIL ถ้าน้อยกว่า Threshold, หรือ None หากภาพมีปัญหาจากการเช็คอื่นๆ ซึ่งอาจทำให้การคำนวณค่าความคมชัดเพี้ยนไปได้ เช่น กรณีที่ภาพมืดเกินไป หรือมีคอนทราสต์ต่ำเกินไป และจะถูกแปลผลในตอนสุดท้ายว่าเป็นเฟรม SKIP

check_occlusion

ไฟล์ qc/checks/check_occlusion.py ฟังก์ชัน _skin_ratio_ycrcb

```
def _skin_ratio_ycrcb(
    ycrcb_roi: np.ndarray,
    cr_bounds: Tuple[int, int],
    cb_bounds: Tuple[int, int],
) -> float:
    """Fraction of pixels in the ROI whose Cr/Cb fall within the skin bounds."""
```

```

if ycrcb_roi.size == 0:
    return 0.0
cr = ycrcb_roi[:, :, 1]
cb = ycrcb_roi[:, :, 2]
cr_lo, cr_hi = cr_bounds
cb_lo, cb_hi = cb_bounds
mask = (cr >= cr_lo) & (cr <= cr_hi) & (cb >= cb_lo) & (cb <= cb_hi)
return float(np.count_nonzero(mask)) / float(mask.size)

```

ไฟล์ `gc/checks/check_occlusion.py` ฟังก์ชัน `check_occlusion`

```

# --- decide: any REQUIRED region below the skin-ratio floor is occluded ---
occluded: List[str] = []
for region in req:
    r = ratios.get(region)
    # A region we could not build an ROI for (missing landmarks) is treated
    # as occluded for a REQUIRED region - we cannot confirm skin is present.
    if r is None or r < min_skin_ratio:
        occluded.append(region)

```

สอดคล้องกับความต้องการระบบ “ห้ามมีวัตถุบังใบหน้า ตั้งแต่หน้าผากจนถึงลำคอ และสวมอุปกรณ์ที่บังใบหน้า เช่น ไม่มีผมยาวบังหน้าหรือตา ไม่สวมหน้ากาก ไม่สวมแว่นกันแดด ไม่สวมหมวก” เช็คว่ามีวัตถุบังใบหน้าหรือไม่ โดยใช้หลักการเช็คค่าสัดส่วนพิกเซลสีผิวแทนหากบริเวณที่ควรเป็นผิวหนังกลับไม่ปรากฏสีผิวจะแปลว่ามีสิ่งบังอยู่ โดยไม่ได้ใช้การตรวจจับวัตถุโดยใช้ pretrained model เหมือนในกรณี walk เนื่องจากโดยธรรมชาติกรอบใบหน้าที่ค่อนข้างเล็ก(ถ้าเทียบกับกรอบที่ใช้ในการเช็คอื่นๆ เช่น walk วัตถุที่อาจนำมาบังใบหน้าที่มีความหลากหลายเกินกว่าจะระบุเป็นรายชนิดได้ จึงมีได้นำ YOLO มาใช้ในส่วนการตรวจจับวัตถุบัง

ขั้นตอนการทำงาน คือ กำหนดบริเวณที่ต้องตรวจสอบจาก landmark ได้แก่ ตาซ้าย ตาขวา จมูก และปาก ตามค่า `required_regions` ใน `config.yml` แล้วแปลงภาพเป็นปริภูมิสี YCrCb แล้วนับสัดส่วนพิกเซลที่มีค่า Cr อยู่ในช่วง 133–173 และค่า Cb อยู่ในช่วง 77–127 ซึ่งถือว่าเป็นช่วงของสีผิวมนุษย์ หากบริเวณใดมีสัดส่วนพิกเซลสีผิวต่ำกว่าเกณฑ์ `min_skin_ratio` (0.40 ตามค่าใน `config.yml`) จะถือว่าบริเวณนั้นถูกบัง และคืนค่ากลับเป็น FALSE พร้อมระบุเหตุผลชื่อบริเวณที่มีปัญหา

5.2.3 การเช็คระดับ sequence

เป็นการเช็คเพื่อตรวจสอบว่าอาสาสมัครทำท่าทางตามที่กำหนดในเอกสารความต้องการระบบหรือไม่ สอดคล้องกับส่วนนี้

“ในภาพวิดีโอ จะต้องเห็นเต็มศีรษะจนถึงคอ และหันศีรษะซ้ายๆ ดังนี้ เริ่มจากหน้าตรงนับ 5 วินาที, หันซ้าย 90 องศาจนเห็นหูค้างไว้ 2 วินาที, หันกลับมาหน้าตรงค้างไว้ 2 วินาที, หันขวา 90 องศาจนเห็นหูค้างไว้ 2

วินาที, หันกลับมาหน้าตรงค้างไว้ 2 วินาที, ก้มหน้าคางชิดคอค้างไว้ 2 วินาที, หันกลับมาหน้าตรงค้างไว้ 2 วินาที, เงยหน้ามองเพดานค้างไว้ 2 วินาที, หันกลับมาหน้าตรงค้างไว้ 2 วินาที ซึ่งต้องทำต่อเนื่อง”

การจำแนกท่าทางในแต่ละเฟรม

ไฟล์ `qc/checks/_turn_common.py` ฟังก์ชัน `classify_frame`

```
def classify_frame(entry: dict, th: TurnThresholds) -> str:
    """Map ONE timeline entry to a position label.

    Returns one of: FRONT, LEFT, RIGHT, DOWN, UP, GAP, NEUTRALish.
    This is the single source of per-frame meaning shared by both versions.
    """
    if not entry.get("face_detected"):
        return GAP

    yaw = entry.get("yaw")
    pitch = entry.get("pitch")
    if yaw is None or pitch is None:
        # Detected but pose failed – treat like a gap (no usable angle).
        return GAP

    # FRONT: neutral on BOTH axes (tight zone).
    if abs(yaw) < th.front_zone_yaw and abs(pitch) < th.front_zone_pitch:
        return FRONT

    # Turn classification: pick the axis that is most strongly past its floor.
    # A frame is rarely both a big yaw and a big pitch in this protocol (turns
    # are done one axis at a time), but if it is, the larger normalized
    # excursion wins, so the label reflects the dominant motion.
    yaw_excess = (abs(yaw) - th.yaw_turn_floor)
    pitch_excess = (abs(pitch) - th.pitch_turn_floor)

    yaw_is_turn = yaw_excess >= 0
    pitch_is_turn = pitch_excess >= 0

    if yaw_is_turn and (not pitch_is_turn or yaw_excess >= pitch_excess):
        return LEFT if yaw < 0 else RIGHT
    if pitch_is_turn:
        return DOWN if pitch < 0 else UP

    # Detected, but in the dead-band on both axes: neither front nor a turn.
    return NEUTRALish
```

จำแนกท่าทางของแต่ละเฟรมจากมุม yaw การหันซ้ายขวา และ pitch การก้มเงย ที่คำนวณมาจาก landmark ของใบหน้า รายงานออกเป็น 7 แบบได้แก่ FRONT, LEFT, RIGHT, DOWN, UP, GAP (กรณีที่ตรวจไม่พบใบหน้า) และ NEUTRALish (พบใบหน้าแต่มุมค่อนข้างกำกวม ตัดสินไม่ได้ว่าใบหน้าที่กำลังหันไปในทางใด) ในกรณีที่เฟรมหนึ่งมีทั้งมุม yaw และ pitch เกินเกณฑ์พร้อมกัน ระบบจะเลือกแกนที่เกินเกณฑ์ไปมากกว่าเป็นตัวกำหนดท่าทาง

`check_turn_left`, `check_turn_right`, `check_turn_down`, `check_turn_up`, `check_turn_front`

ไฟล์ `qc/checks/check_turn_sequence_seg.py` ฟังก์ชัน `check_turn_sequence_seg`

```
def check_turn_sequence_seg(timeline, config, *, sample_fps=1.0, emit_row=None):
    ...
```

ตรวจสอบว่าอาสาสมัครทำท่าทางแต่ละท่าครบถ้วนหรือไม่ โดยมีเงื่อนไข 2 ประการ คือ ตรวจพบท่าทางนั้นในวิดีโอ และ ค้างท่าไว้นานเพียงพอคือมากกว่าหรือเท่ากับ 2 วินาทีสำหรับทุกท่า ยกเว้น FRONT ในจังหวะแรกสุดของวิดีโอต้องค้างท่านาน 5 วินาที โดยระบบจะรวมเฟรมที่มีท่าทางเดียวกันติดต่อกันเป็นช่วงเดียวกัน แล้วคำนวณระยะเวลาของแต่ละช่วงจากจำนวนเฟรมหารด้วยอัตราการสุ่มเฟรมจากวิดีโอตัวอย่าง หากช่วงใดสั้นกว่าเกณฑ์จะรายงานเป็น FAIL พร้อมระบุเหตุผลเป็นระยะเวลาที่วัดได้ (เช่น down detected but hold too short: 1.8s < 2.0s)

check_turn_sequence

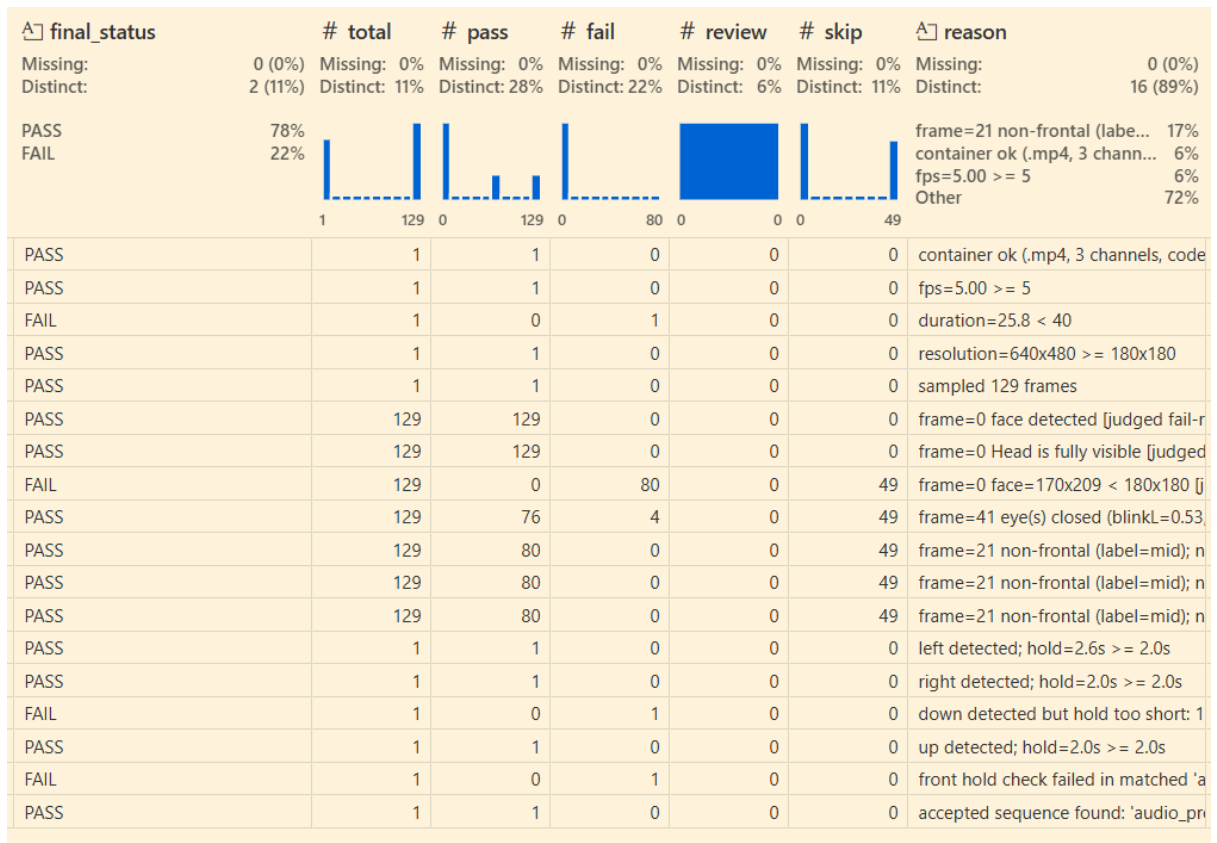
ไฟล์ qc/checks/check_turn_sequence_seg.py ฟังก์ชัน check_turn_sequence_seg

```
def check_turn_sequence_seg(timeline, config, *, sample_fps=1.0, emit_row=None):
    ...
```

ลำดับที่ถูกต้องของการหันคือ front -> left -> front -> right -> front -> up -> front -> down -> front จึงต้องมีการตรวจสอบลำดับการหันดังกล่าว ตัวอย่างผลลัพธ์การหัน เช่น check_turn_sequence ได้ผลเป็น PASS ในขณะที่ check_turn_front และ check_turn_down ได้ผลเป็น FAIL ซึ่งไม่ขัดแย้งกันเนื่องจากอาสาสมัครทำท่าทางถูกลำดับแต่ค้างในแต่ละท่าไม่นานพอ

ตัวอย่างผลลัพธ์การรัน face_001_result.csv

#	volunteer_id	data_type	filename	check_level	check_name	final_status
Missing:	0 (0%)	Missing:	0 (0%)	Missing:	0 (0%)	Missing:
Distinct:	1 (6%)	Distinct:	1 (6%)	Distinct:	3 (17%)	Distinct:
		face_rgb	100% 001_face_rgb.mp4	100% frame sequence video	39% 33% 28% 18 Distinct values	0 (0%) Missing: 18 (100%) Distinct: PASS FAIL
	Min 1	Max 1				
0	1	face_rgb	001_face_rgb.mp4	video	check_container	PASS
1	1	face_rgb	001_face_rgb.mp4	video	check_fps	PASS
2	1	face_rgb	001_face_rgb.mp4	video	check_duration	FAIL
3	1	face_rgb	001_face_rgb.mp4	video	check_resolution	PASS
4	1	face_rgb	001_face_rgb.mp4	video	frames_sampled	PASS
5	1	face_rgb	001_face_rgb.mp4	frame	check_face_detected	PASS
6	1	face_rgb	001_face_rgb.mp4	frame	check_head_fully	PASS
7	1	face_rgb	001_face_rgb.mp4	frame	check_face_size	FAIL
8	1	face_rgb	001_face_rgb.mp4	frame	check_eyes_open	PASS
9	1	face_rgb	001_face_rgb.mp4	frame	check_brightness	PASS
10	1	face_rgb	001_face_rgb.mp4	frame	check_face_blur	PASS
11	1	face_rgb	001_face_rgb.mp4	frame	check_occlusion	PASS
12	1	face_rgb	001_face_rgb.mp4	sequence	check_turn_left	PASS
13	1	face_rgb	001_face_rgb.mp4	sequence	check_turn_right	PASS
14	1	face_rgb	001_face_rgb.mp4	sequence	check_turn_down	FAIL
15	1	face_rgb	001_face_rgb.mp4	sequence	check_turn_up	PASS
16	1	face_rgb	001_face_rgb.mp4	sequence	check_turn_front	FAIL
17	1	face_rgb	001_face_rgb.mp4	sequence	check_turn_sequence	PASS



5.3 การตรวจสอบข้อมูลฝ่ามือ

ไฟล์ : qc/pipelines/palm.py

สั่งรันตรวจสอบข้อมูลชีวมิติประเภทใบหน้า

ไฟล์ : run_palm.py

ประกอบไปด้วยการเช็คหลายระดับ สถานะผลลัพธ์การเช็คที่มีได้คือ PASS SKIP FAIL โดย PASS แปลว่าการเช็คผ่าน SKIP ความหมายคือจะไม่ทำการเช็คในภาพนั้นๆ FAIL แปลว่าการเช็คไม่ผ่าน

5.3.1 การเช็คระดับ image

check_container, check_resolution

ไฟล์ qc/check_metadata.py ฟังก์ชัน check_container

```
def check_container(meta, *, require_rgb: bool = True, expected_ext: str = ".mp4"):
    """Is the file a readable .mp4, and (for rgb) 3-channel color?"""
    if not meta.readable:
        return (FAIL, f"unreadable: {meta.reason}")
    if meta.extension != expected_ext:
        return (FAIL, f"extension={meta.extension} != {expected_ext}")
    if require_rgb:
        if meta.channel_count is None or meta.channel_count < 3:
            return (FAIL, "channel count unknown (cannot verify RGB)")
        if not meta.is_color:
            return (FAIL, "grayscale content (channels identical; not true RGB)")
    return (PASS, f"container ok ({meta.extension}, "
                f"{meta.channel_count} channels, codec={meta.codec_fourcc})")
```

ไฟล์ qc/check_metadata.py ฟังก์ชัน check_resolution

```
def check_resolution(meta, *, min_width: int = 180, min_height: int = 180):
    if meta.width is None or meta.height is None:
        return (FAIL, "resolution unknown (cannot verify against spec)")
    if meta.width >= min_width and meta.height >= min_height:
        return (PASS, f"resolution={meta.width}x{meta.height} "
                    f">= {min_width}x{min_height}")
    return (FAIL, f"resolution={meta.width}x{meta.height} "
                f"< {min_width}x{min_height}")
```

สอดคล้องกับความต้องการระบบที่ว่า

"รูปแบบการบันทึกชื่อไฟล์ ตัวอย่างเช่น 0001_palm_L_N.jpg"

"ขนาดกว้างยาวไม่น้อยกว่า 200 x 200 พิกเซล (pixel) ตั้งแต่กลางนิ้วจนถึงข้อมือ "

การเช็คทั้งสองรายการนี้ใช้ฟังก์ชันเดียวกันกับการตรวจสอบข้อมูลใบหน้า

(qc/checks/check_metadata.py) ซึ่งได้อธิบายรายละเอียดการทำงานไว้แล้วในหัวข้อ 5.2.1 โดยฟังก์ชัน

probe_image ใน qc/utils/image.py จะคืนค่ากลับเป็น instance ที่มีรูปร่างเดียวกันกับที่ probe_video

คืนค่ากลับจึงสามารถนำฟังก์ชันเดิมของ face ที่ใช้เช็ค metadata มาใช้แต่ปรับค่าของ argument เช่นให้
expected_ext = “.jpg”

check_palm_present

ไฟล์ qc/pipelines/palm.py ซึ่งเรียกใช้ฟังก์ชัน create_hand_landmarker, detect_hand จาก

qc/checks/hand_landmarker.py

```
detector = create_hand_landmarker(  
    model_path=hl_cfg.get("model_path", "models/hand_landmarker.task"),  
    num_hands=hands_cfg.get("max_num_hands", 1),  
    min_hand_detection_confidence=hands_cfg.get(  
        "min_detection_confidence", 0.5),  
)  
try:  
    hand_result = detect_hand(path, detector=detector)  
finally:  
    detector.close()  
  
if hand_result.ok:  
    add("check_palm_present", "PASS", hand_result.message, level="image")  
else:  
    msg = hand_result.message or "No hand detected"  
    status = multi_policy if msg.startswith("Multiple hands") else "FAIL"  
    add("check_palm_present", status, msg, level="image")
```

ฟังก์ชัน detect_hand จะทำการตรวจจับมือจากการเรียกใช้โมเดล MediaPipe Hand Landmarker ผ่าน
การใช้ Tasks API และโปรแกรมจะเช็คผลที่ได้รับกลับมาผ่านทาง instance type HandResult ซึ่ง
ประกอบด้วย landmarks ของมือจำนวน 21 จุด

```
@dataclass  
class HandResult:  
    """One image's hand detection, carrying every representation the palm  
    checks consume so each check changes minimally. Shaped to mirror  
    FaceResult (face_landmarker.py)."""  
    ok: bool  
    message: str  
    landmarks_px: Optional[list] = None # [(x, y, z)] pixel space  
    landmarks_norm: Optional[Any] = None # raw normalized landmark list (.x/.y/.z)  
    bbox: Optional[tuple] = None # (x, y, w, h) px, 10% margin  
    norm_box: Optional[tuple] = None # (x, y, w, h) normalized  
    handedness: Optional[str] = None # "Left" / "Right"  
    handedness_score: Optional[float] = None # confidence of the handedness label  
    world_landmarks: Optional[Any] = None # raw world landmark list (.x/.y/.z, meters)
```

การเช็คนี้เป็นการเช็คลำดับแรกของการเช็คฝ่ามือ การเช็คฝ่ามือรายการอื่นอาจต้องอาศัยผลลัพธ์ PASS ของ
การเช็คนี้ หากตรวจไม่พบมือในภาพใด การเช็คอื่นๆ ในภาพนั้นจะทำงานไม่ได้ (อาจได้ผล PASS FAIL ที่ผิด)
จึงต้องให้ผลลัพธ์การเช็คอื่นๆ บันทึกเป็น SKIP แปลว่าจะข้ามการเช็คนั้นไป นอกจากนี้ในกรณีที่ตรวจพบมือ
มากกว่า 1 มือในภาพเดียว จะได้สถานะ FAIL ตามค่า multiple_hands_policy = FAIL ใน config.yml

check_palm_handedness

ไฟล์ `qc/checks/check_palm_handedness.py`

```
def check_palm_handedness(
    handedness,
    filename_hand,
    *,
    handedness_score=None,
    expected_relation: str = "same",
    min_confidence: float = 0.7,
):
    fh = (filename_hand or "").upper()
    score_txt = f"{handedness_score:.2f}" if handedness_score is not None else "n/a"

    if fh not in ("L", "R"):
        return (None, f"handedness not graded: unknown filename hand "
                    f"'{filename_hand}' (mediapipe={handedness}, "
                    f"score={score_txt})")

    mp = _FULL.get((handedness or "").upper())
    if mp is None:
        return (None, f"handedness not graded: no MediaPipe label "
                    f"(filename={fh}, mediapipe={handedness}, score={score_txt})")

    expected = _expected_label(fh, expected_relation)
```

เชื่อว่าข้างซ้ายหรือขวาของมือที่ตรวจจับได้ตรงกับข้างมือที่ระบุไว้ในชื่อไฟล์หรือไม่ เพื่อตรวจกรณีที่อาสาสมัครส่งไฟล์ผิดข้าง เช่น ถ่ายภาพมือขวาแต่บันทึกชื่อไฟล์เป็น 001_palm_L_N.jpg โดยแปลผล PASS FAIL จาก label ที่ MediaPipe ตรวจจับได้เทียบกับข้างมือที่อ่านได้จากชื่อไฟล์ (L / R) ถ้า label ตรงกัน ค็นค่ากลับเป็น TRUE (PASS) แต่ถ้า label ไม่ตรงกันต้องแยกพิจารณาว่าหากคะแนนความมั่นใจที่ได้จากโมเดลต่ำกว่าเกณฑ์ค็นค่ากลับเป็น None (เขียน output ออกมาเป็น SKIP) เนื่องจาก MediaPipe ยังไม่มั่นใจในการจำแนกข้างมือ และถ้าหากคะแนนความมั่นใจของโมเดลถึงเกณฑ์ค็นค่ากลับเป็น FALSE (FAIL) เนื่องจากมีความเป็นไปได้ว่าอาสาสมัครส่งไฟล์ผิดข้างจริง

การใช้ชิ้นงานเฉพาะกับท่า N เท่านั้น ส่วนท่า RL, RR, PU, PD จะได้สถานะ SKIP เนื่องจากพบว่าการเอียงทำให้ MediaPipe จำแนกข้างมือผิดพลาดได้ ตัวอย่างเช่น จากที่ได้มีการทดลองกับข้อมูลบางส่วนพบว่ามือซ้ายข้างเดียวกันถูกจำแนกเป็น Left ในท่า N แต่ถูกจำแนกเป็น Right ในท่า PU

check_palm_size

ไฟล์ `qc/checks/check_palm_size.py`

```
def check_palm_min_size(bbox, min_width: int = 200, min_height: int = 200):
    if bbox is None:
        return (False, "No bounding box provided")
```

```
x, y, w, h = bbox
```

```
# Spec says "not less than", so >= (boundary value passes).
```

```
if w >= min_width and h >= min_height:
```

```
    return (True, f"palm={w}x{h} >= {min_width}x{min_height}")
```

```
    return (False, f"palm={w}x{h} < {min_width}x{min_height}")
```

สอดคล้องกับความต้องการระบบที่ว่า "ขนาดของบริเวณฝ่ามือวัดจากกลางนิ้วถึงข้อมือไม่น้อยกว่า 200 x 200 pixel" จะใช้ขนาด bounding box (bbox) ของมือที่ตรวจจับและคำนวณได้ โดยคำนวณ bbox จากขนาดความกว้างยาวของ landmarks ทั้ง 21 จุดที่ตรวจจับได้ แล้วบวก margin 10% หากไม่พบ bbox ของมือ (กรณีที่ check_palm_present FAIL ไปแล้ว) การใช้นี้จะได้สถานะ SKIP เพื่อไม่ให้เช็คซ้ำซ้อนกัน

check_palm_brightness

ไฟล์ `qc/checks/check_brightness.py`

```
def check_brightness(
    image: Any,
    bbox: Tuple[int, int, int, int],
    dark_threshold: float = 35.0,
    bright_threshold: float = 200.0,
    margin: float = 0.1,
    *,
    input_color_space: Literal["BGR", "RGB"] = "BGR",
) -> Tuple[bool, str]:
    img = _to_bgr(image, input_color_space)
    if img is None or getattr(img, "size", 0) == 0:
        return (False, "invalid_image")

    h_img, w_img = img.shape[:2]
    x, y, bw, bh = bbox

    # Central region: trim `margin` off each side of the supplied bbox
    x_start = max(0, int(x + bw * margin))
    y_start = max(0, int(y + bh * margin))
    x_end = min(w_img, int(x + bw * (1 - margin)))
    y_end = min(h_img, int(y + bh * (1 - margin)))
    if x_end <= x_start or y_end <= y_start:
        return (False, "invalid_crop")

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    region_v = hsv[y_start:y_end, x_start:x_end, 2]
    if region_v.size == 0:
        return (False, "empty_region")

    brightness = float(np.mean(region_v))

    if brightness < dark_threshold:
        status = "too_dark"
    elif brightness > bright_threshold:
        status = "too_bright"
    else:
        status = "normal"

    b = int(round(brightness))
    lo = int(round(dark_threshold))
    hi = int(round(bright_threshold))
```

```

if status == "normal":
    msg = f"brightness={b}; normal; {lo} <= {b} <= {hi}"
elif status == "too_dark":
    msg = f"brightness={b} < {lo}; too_dark"
else: # too_bright
    msg = f"brightness={b} > {hi}; too_bright"

```

```

return (status == "normal", msg)

```

เช็คความสว่างของภาพบริเวณฝ่ามือ โดยใช้ฟังก์ชันเดียวกันกับการเช็คความสว่างของใบหน้า

(check_brightness) ซึ่งได้อธิบายรายละเอียดการทำงานไว้แล้วในหัวข้อ 5.2.2 ความแตกต่างอยู่ที่บริเวณที่

วัด โดยการเช็คใบหน้าวัดภายใน bbox ของใบหน้า ส่วนการเช็คฝ่ามือวัดภายใน bbox ของมือที่ได้จาก

detect_hand

check_palm_spread

ไฟล์ qc/checks/check_palm_spread.py

```

def check_palm_spread(
    landmarks_norm,
    *,
    min_gap_inter_tip_deg: float = _DEFAULT_MIN_GAP_INTER_TIP,
    min_gap_thumb_tip_deg: float = _DEFAULT_MIN_GAP_THUMB_TIP,
    min_gap_inter_pip_deg: float = _DEFAULT_MIN_GAP_INTER_PIP,
    min_gap_thumb_pip_deg: float = _DEFAULT_MIN_GAP_THUMB_PIP,
    required_pairs=None,
    frame_margin: float = _DEFAULT_FRAME_MARGIN,
):
    ok, info = calculate_finger_gaps(
        landmarks_norm, frame_margin=frame_margin)
    if not ok:
        return (False, info.get("error", "Could not compute finger spread"))

    source: str = info["source"]
    gaps = info["gaps"]
    req = list(required_pairs) if required_pairs else list(_DEFAULT_REQUIRED_PAIRS)

    if source == "tip":
        thumb_floor, inter_floor = min_gap_thumb_tip_deg, min_gap_inter_tip_deg
    else:
        thumb_floor, inter_floor = min_gap_thumb_pip_deg, min_gap_inter_pip_deg

    closed = []
    for pair in req:
        g = gaps.get(pair)
        if g is None or g < _floor(pair):
            closed.append(pair)

```

สอดคล้องกับความต้องการที่ว่า “ขนาดกว้างยาวไม่น้อยกว่า 200 x 200 พิกเซล (pixel) ตั้งแต่กลางนิ้วจนถึงข้อมือ และอยู่ในท่ากางฝ่ามือและนิ้วทุกนิ้ว ให้ภาพเห็นทั้งฝ่ามือ นิ้วมือทั้งห้ากางออกตามธรรมชาติ ไม่ชิดติดกันจนบังแสง และไม่ออกแรงเกร็งจนฝ่ามือแอ่นตัว” จะเช็คค่าอาสามัครกางนิ้วมือออกจากกันหรือไม่ โดยวัดจากมุมระหว่างนิ้วที่อยู่ติดกัน ซึ่งคำนวณจากเวกเตอร์ที่ลากจากข้อโคนนิ้ว (MCP) ไปยังปลายนิ้ว โดยมี

ข้อยกเว้นคือหากมีปลายนิ้วใดอยู่นอกเฟรมจะเปลี่ยนไปใช้वेक्टरที่ลากจากข้อโคนนิ้ว (MCP) ไปยังข้อกลางนิ้ว (PIP) เป็นจุดอ้างอิงแทน แล้วหามุมระหว่างเวกเตอร์ของนิ้วสองนิ้วที่อยู่ติดกัน

มุมระหว่างนิ้วที่ต้องตรวจสอบมี 4 คู่ ได้แก่ นิ้วโป้ง-นิ้วชี้, นิ้วชี้-นิ้วกลาง, นิ้วกลาง-นิ้วนาง และ นิ้วนาง-นิ้วก้อย โดยหากมุมของนิ้วใดคู่หนึ่งน้อยกว่าเกณฑ์จะคืนค่ากลับเป็น TRUE แต่ถ้ามีมุมของนิ้วใดคู่หนึ่งน้อยกว่าเกณฑ์จะคืนค่ากลับเป็น FALSE พร้อมระบุว่านิ้วใดชิดกันเกินไปและวัดค่าได้เท่าใด โดยผู้ใช้สามารถปรับเกณฑ์มุมเหล่านี้ได้ใน config.yml โดยการเช็คการกางนิ้วทำงานเฉพาะกับท่า N เท่านั้น เช่นเดียวกับ check_palm_handedness ส่วนในท่าอื่นๆ ของมือจะให้ผลเป็น SKIP แทน

check_palm_fully

ไฟล์ qc/checks/check_palm_fully.py

```
_WRIST_IDX = 0
_MIDDLE_FINGER_MCP_IDX = 9

def check_palm_fully(landmarks, image_height: int, image_width: int,
                     margin_px: int = 10):
    if not landmarks:
        return (False, "No landmarks provided")

    try:
        wrist_x, wrist_y = landmarks[_WRIST_IDX][0], landmarks[_WRIST_IDX][1]
        mcp_x = landmarks[_MIDDLE_FINGER_MCP_IDX][0]
        mcp_y = landmarks[_MIDDLE_FINGER_MCP_IDX][1]
    except (IndexError, TypeError):
        return (False, "Landmark list missing required points")

    wrist_top_cut = wrist_y < margin_px
    wrist_bottom_cut = wrist_y > image_height - margin_px
    wrist_left_cut = wrist_x < margin_px
    wrist_right_cut = wrist_x > image_width - margin_px

    mcp_top_cut = mcp_y < margin_px
    mcp_bottom_cut = mcp_y > image_height - margin_px
    mcp_left_cut = mcp_x < margin_px
    mcp_right_cut = mcp_x > image_width - margin_px

    if wrist_bottom_cut and mcp_top_cut:
        return (False, "Wrist and upper palm might be cut")
    if wrist_bottom_cut or wrist_top_cut:
        return (False, "Wrist might be cut")
    if mcp_top_cut or mcp_bottom_cut:
        return (False, "Upper palm might be cut")
    if wrist_left_cut or mcp_left_cut:
        return (False, "Left side of hand might be cut")
    if wrist_right_cut or mcp_right_cut:
        return (False, "Right side of hand might be cut")
    return (True, "Palm is fully visible")
```

เชื่อว่าฝ่ามืออยู่ในกรอบภาพครบถ้วนหรือไม่ โดยใช้ landmark 2 จุดที่เป็นขอบเขตตามแนวยาวของมือ ได้แก่ ข้อมือ (จุดที่ 0) ซึ่งเป็นส่วนฐาน และ ข้อโคนนิ้วกลาง (จุดที่ 9) ซึ่งเป็นส่วนสำคัญที่ต้องมีครอบคลุมเพื่อใช้ในการคำนวณเวกเตอร์เพื่อประเมินองศาการหันมือในเชิงถัดไป หากจุดใดอยู่ใกล้ขอบภาพเกินกว่าค่า margin_px (ซึ่งมีค่า default ที่ 10 พิกเซล) จะถือว่าส่วนนั้นถูกตัดออกจากเฟรม และคืนค่ากลับเป็น FALSE แต่หากทั้งสองจุดอยู่ในกรอบภาพจะคืนค่ากลับเป็น TRUE

check_palm_angle

ไฟล์ qc/checks/check_palm_angle.py ฟังก์ชัน calculate_palm_angles

```
p0 = _pt(_WRIST)          # wrist
p5 = _pt(_INDEX_MCP)      # index MCP (thumb/index side)
p9 = _pt(_MIDDLE_MCP)     # middle MCP (central forward axis)
p13 = _pt(_RING_MCP)      # ring MCP
p17 = _pt(_PINKY_MCP)     # pinky MCP

forward = p9 - p0
forward_xy = math.hypot(float(forward[0]), float(forward[1]))
if forward_xy < 1e-9:
    return (False, {"error": "degenerate palm geometry (zero-length wrist-to-middle axis)"})
pitch = math.degrees(math.atan2(-float(forward[2]), forward_xy))

# ---- Roll: thumb/index side (p5) vs pinky side mean(p13,p17), depth-wise
thumb_side = p5
pinky_side = np.mean([p13, p17], axis=0)
side_vec = pinky_side - thumb_side
side_xy = math.hypot(float(side_vec[0]), float(side_vec[1]))
if side_xy < 1e-9:
    return (False, {"error": "degenerate palm geometry (zero-width palm side axis)"})

raw_roll_degree = math.degrees(math.atan2(float(side_vec[2]), side_xy))
hs = str(handedness or "").strip().upper()
is_right = hs in ("R", "RIGHT")
# Exact roll convention
# RL reads POSITIVE, RR reads NEGATIVE, consistent across L and R.
# _POSE_SIGN is updated to {"RL":+1,"RR":-1} to match. (Option A.)
roll = -raw_roll_degree if is_right else raw_roll_degree

up_axis = _unit(forward, "up axis")
side_axis = _unit(side_vec, "side axis")
normal = np.cross(side_axis, up_axis)
normal = _unit(normal, "debug normal")

return (True, {
    "roll": float(roll),
    "pitch": float(pitch),
    "normal": (float(normal[0]), float(normal[1]), float(normal[2])),
    "raw_side_depth_deg": float(raw_roll_degree),
    "raw_up_depth_deg": float(-forward[2]),
    "handedness_used": "R" if is_right else "L_or_default",
})
```

สอดคล้องกับความต้องการระบบ “ต้องเก็บภาพถ่ายเส้นเลือดฝ่ามือทั้งซ้ายและขวา ข้างละ 5 มุม โดยให้หมุนข้อมือตามแกนในภาพที่ 1 โดยมุมข้อมือแสดงอยู่ในภาพที่ 2 ตามลำดับ” การหมุนมือต้องทำทั้งมือซ้ายและมือขวา โดยในมือแต่ละข้างต้องทำ 5 ท่าได้แก่ Neutral (N), Roll Left (RL), Roll Right (RR), Pitch Up (PU), Pitch Down (PD) โดยในแต่ละท่าการหมุนต้องหมุนข้อมือเพียงเล็กน้อยไม่เกินมุม 45 องศา

มุม roll และ pitch อาศัยค่าพิกัด z ของ landmark ซึ่ง MediaPipe คำนวณมาให้เป็นค่าในรูปแบบที่มีการทำ normalized มาแล้ว โดยมีข้อมือเป็นจุดกำเนิด และค่าที่น้อยกว่าหมายถึงอยู่ใกล้กล้องมากกว่า การวัดมุมมีหลักการคือ วาดเวกเตอร์ แล้วเทียบมุมระหว่างพิกัดแกน z กับแกน xy ของเวกเตอร์นั้นๆ โดยมุม pitch วัดจากแกนที่ลากจากข้อมือ (จุดที่ 0) ไปยังข้อโคนนิ้วกลาง (จุดที่ 9)

- หากข้อมืออยู่ใกล้กล้องมากกว่าปลายมือ จะได้ค่า pitch เป็นลบ (ท่า PU)
- หากด้านปลายนิ้วอยู่ใกล้กล้องมากกว่าปลายมือ จะได้ค่า pitch เป็นบวก (ท่า PD)

มุม roll วัดจากแกนที่ลากจากด้านนิ้วโป้ง (ข้อโคนนิ้วชี้ จุดที่ 5) ไปยังด้านนิ้วก้อย (ค่าเฉลี่ยของข้อโคนนิ้วนาง จุดที่ 13 และข้อโคนนิ้วก้อย จุดที่ 17)

- ท่า RL จะได้ค่า roll เป็นบวก
- ท่า RR จะได้ค่า roll เป็นลบ

ไฟล์ `qc/checks/check_palm_angle.py` ฟังก์ชัน `calculate_palm_angles`

```
def check_palm_pose_absolute(
    pose: str,
    hand: str,
    pose_angles: dict,
    *,
    max_abs_deg: float = 45.0,
    min_rotation_deg: float = 10.0,
    neutral_tol_deg: float = 10.0,
    hand_sign_overrides: Optional[dict] = None,
):
    """Grade one palm image on its OWN raw roll/pitch (no N baseline, no delta).

    Args:
        pose: "N" | "RL" | "RR" | "PU" | "PD".
        hand: "L" | "R" (from filename) -- selects the roll sign convention.
        pose_angles: {"roll":..., "pitch":...} measured for THIS image.
        max_abs_deg: upper band bound (spec: 45).
        min_rotation_deg: a rotated pose must reach at least this magnitude.
        neutral_tol_deg: for N, both axes must be within +/- this.
        hand_sign_overrides: optional per-(hand,pose) sign flips. See _band_for_pose.

    Returns:
        (success, message). PASS/FAIL only. A rotated pose's OTHER axis is
        reported but never gates the verdict.
    """
```

```

pose = (pose or "").upper()
hand = (hand or "").upper()

try:
    roll = float(pose_angles["roll"])
    pitch = float(pose_angles["pitch"])
except (KeyError, TypeError, ValueError) as e:
    return (False, f"missing angle data: {e}")

# --- N (neutral): both axes near zero ---
if pose == "N":
    roll_ok = abs(roll) <= neutral_tol_deg
    pitch_ok = abs(pitch) <= neutral_tol_deg
    if roll_ok and pitch_ok:
        return (True,
                f"{hand}/N ok: raw roll={roll:+.1f}, pitch={pitch:+.1f} "
                f"within +/-{neutral_tol_deg:g}")
    bad = []
    if not roll_ok:
        bad.append(f"roll={roll:+.1f} not within +/-{neutral_tol_deg:g}")
    if not pitch_ok:
        bad.append(f"pitch={pitch:+.1f} not within +/-{neutral_tol_deg:g}")
    return (False, f"{hand}/N fail (not neutral): {'', '.join(bad)}")

axis = _POSE_AXIS.get(pose)
if axis is None:
    return (False, f"unknown pose '{pose}' (expected N/RL/RR/PU/PD)")

_, lo, hi = _band_for_pose(
    hand, pose,
    max_abs_deg=max_abs_deg, min_rotation_deg=min_rotation_deg,
    neutral_tol_deg=neutral_tol_deg, hand_sign_overrides=hand_sign_overrides)

active = roll if axis == "roll" else pitch
active_name = axis # "roll" or "pitch"
other = pitch if axis == "roll" else roll
other_name = "pitch" if axis == "roll" else "roll"

in_band = lo <= active <= hi

if in_band:
    return (True,
            f"{hand}/{pose} ok: raw {active_name}={active:+.1f} "
            f"within [{lo:g},{hi:g}], {other_name}={other:+.1f}")

return (False,
        f"{hand}/{pose} fail: raw {active_name}={active:+.1f} "
        f"not in [{lo:g},{hi:g}], {other_name}={other:+.1f}")

```

เมื่อได้มุมแล้ว ฟังก์ชัน check_palm_pose_absolute ทำหน้าที่ตัดสินว่ามุมที่วัดได้ตรงกับท่าที่ระบุในชื่อไฟล์หรือไม่ แต่ละภาพถูกตัดสินว่า PASS หรือ FAIL ด้วยค่ามุมของแต่ละท่าทางในรูปเทียบกับค่าในช่วงที่กำหนดไว้ โดยค่าของช่วงสามารถปรับแก้ได้ใน config.yml โดยค่าที่ใช้เป็น default คือ ท่า N ต้องมีค่า roll และ pitch ทั้งสองแกน อยู่ในช่วง ± 15 องศา (ค่า config.yml ส่วน neutral_tol_deg) จึงจะผ่าน ส่วนท่า RL, RR, PU, PD ตัดสินจากแกนหลักของท่านั้นเพียงแกนเดียว โดยท่า RL และ RR ตัดสินจากแกน roll

- RL roll +10 ถึง +45 องศา
- RR roll - 45 ถึง -10 องศา

ส่วนทำ PU และ PD ตัดสินจากแกน pitch

- PU pitch -45 ถึง -10 องศา
- PD pitch +10 ถึง +45 องศา

ตัวอย่างผลลัพธ์การรัน palm_096_detail.csv

#	volunteer_id	data_...	filename	check_level	check...	status	reason
Missing:	0 (0%)	Missing: 0%	Missing: 0 (0%)	Missing: 0 (0%)	Missing: 0%	Missing: 0 (0%)	Missing: 0 (0%)
Distinct:	1 (1%)	Distinct: 1%	Distinct: 10 (11%)	Distinct: 1 (1%)	Distinct: 10%	Distinct: 2 (2%)	Distinct: 47 (52%)
		palm 100%	096_palm_L_... 10%	image 100%	check_... 11%	PASS 82%	container ok (jpg, 3 channels, codec=None) 11%
			096_palm_L_... 10%		check_r...11%	SKIP 18%	Hand detected successfully 11%
			096_palm_L_... 10%		check_... 11%		Palm is fully visible 11%
			Other 70%		Other 67%		Other 67%
0	96	palm	096_palm_L_N.jpg	image	check_contair	PASS	container ok (jpg, 3 channels, codec=None)
1	96	palm	096_palm_L_N.jpg	image	check_resolut	PASS	resolution=1477x1108 >= 200x200
2	96	palm	096_palm_L_N.jpg	image	check_palm_p	PASS	Hand detected successfully
3	96	palm	096_palm_L_N.jpg	image	check_palm_h	PASS	handedness ok: filename=L matches mediapipe=Left (score=0.91,
4	96	palm	096_palm_L_N.jpg	image	check_palm_s	PASS	palm=541x585 >= 200x200
5	96	palm	096_palm_L_N.jpg	image	check_palm_b	PASS	brightness=176; normal; 35 <= 176 <= 220
6	96	palm	096_palm_L_N.jpg	image	check_palm_s	PASS	spread ok; source=tip; thumb-index=32.2 index-middle=4.0 middl
7	96	palm	096_palm_L_N.jpg	image	check_palm_fi	PASS	Palm is fully visible
8	96	palm	096_palm_L_RL.jpg	image	check_contair	PASS	container ok (jpg, 3 channels, codec=None)
9	96	palm	096_palm_L_RL.jpg	image	check_resolut	PASS	resolution=1477x1108 >= 200x200
10	96	palm	096_palm_L_RL.jpg	image	check_palm_p	PASS	Hand detected successfully
11	96	palm	096_palm_L_RL.jpg	image	check_palm_h	SKIP	pose=RL not graded (handedness checked on N only; MediaPipe L
12	96	palm	096_palm_L_RL.jpg	image	check_palm_s	PASS	palm=289x609 >= 200x200
13	96	palm	096_palm_L_RL.jpg	image	check_palm_b	PASS	brightness=188; normal; 35 <= 188 <= 220
14	96	palm	096_palm_L_RL.jpg	image	check_palm_s	SKIP	pose=RL not in eval_poses=['N'] (spread enforced on neutral only)
15	96	palm	096_palm_L_RL.jpg	image	check_palm_fi	PASS	Palm is fully visible
80	96	palm	096_palm_L_N.jpg	image	check_palm_a	PASS	L/N ok: raw roll=+1.4, pitch=-3.6 within +/-15
81	96	palm	096_palm_L_RL.jpg	image	check_palm_a	PASS	L/RL ok: raw roll=+38.2 within [10,45], pitch=-10.2
82	96	palm	096_palm_L_RR.jpg	image	check_palm_a	PASS	L/RR ok: raw roll=-25.5 within [-45,-10], pitch=+3.4
83	96	palm	096_palm_L_PU.jpg	image	check_palm_a	PASS	L/PU ok: raw pitch=-43.9 within [-45,-10], roll=-5.2
84	96	palm	096_palm_L_PD.jpg	image	check_palm_a	PASS	L/PD ok: raw pitch=+20.3 within [10,45], roll=+2.4
85	96	palm	096_palm_R_N.JPG	image	check_palm_a	PASS	R/N ok: raw roll=+1.4, pitch=-4.5 within +/-15
86	96	palm	096_palm_R_RL.JPG	image	check_palm_a	PASS	R/RL ok: raw roll=+38.5 within [10,45], pitch=+3.2
87	96	palm	096_palm_R_RR.JPG	image	check_palm_a	PASS	R/RR ok: raw roll=-41.3 within [-45,-10], pitch=-0.3
88	96	palm	096_palm_R_PU.JPG	image	check_palm_a	PASS	R/PU ok: raw pitch=-42.4 within [-45,-10], roll=-2.2
89	96	palm	096_palm_R_PD.JPG	image	check_palm_a	PASS	R/PD ok: raw pitch=+28.5 within [10,45], roll=+9.7

5.4 การตรวจสอบข้อมูลการเดิน

ไฟล์ : `qc/pipelines/walk.py`

สั่งรันตรวจสอบข้อมูลชีวมิติประเภทวิดีโอท่าทางการเดิน

ไฟล์ : `run_walk.py`

ประกอบไปด้วยการเช็คหลากหลายระดับ สถานะผลลัพธ์การเช็คที่มีได้คือ PASS SKIP FAIL โดย PASS แปลว่าการเช็คนั้นผ่าน SKIP ความหมายคือจะไม่ทำการเช็คนั้นในเฟรมนั้นๆ FAIL แปลว่าการเช็คนั้นไม่ผ่าน โดยในแต่ละวิดีโอจะต้องไม่มีสถานะ FAIL แม้แต่การเช็คเดียว มิฉะนั้นจะถือว่าทั้งวิดีโอไม่ผ่านตามข้อกำหนด

การตรวจสอบข้อมูลท่าทางการเดินใช้โมเดล 2 โมเดลร่วมกัน ได้แก่ MediaPipe Pose Landmarker สำหรับตรวจจับบุคคล ซึ่งจะรายงาน 33 landmarks ต่อหนึ่งเฟรม และ YOLOv8-nano สำหรับตรวจจับสิ่งกีดขวาง ในภาพ อาสาสมัครต้องส่งสองไฟล์แยกกัน คือ ไฟล์ F (เช่น 001_walk_F.mp4) สำหรับบันทึกภาพวิดีโอโดยเดินเข้าหากล้องและเดินหันหลังออกจากกล้อง และไฟล์ S (เช่น 001_walk_S.mp4) บันทึกภาพวิดีโอเดินผ่านกล้องไปด้านหลังซ้ายและขวา ทั้งสองไฟล์จะถูกบันทึกภาพพร้อมกัน และการรันเช็คจะทำแยกกันและรายงานผลแยกกันในแต่ละไฟล์

ซึ่งมีการเช็คต่างๆ ดังนี้

5.4.1 การเช็คระดับ video

เป็นการเช็คข้อมูลภาพรวมทั้งวิดีโอ ก่อนเข้าสู่ขั้นตอนการเช็คแต่ละเฟรมของวิดีโอ

`check_container, check_fps, check_duration, check_resolution`

ไฟล์ `utils/video.py` ฟังก์ชัน `probe_video`

ไฟล์ `qc/check_metadata.py`

เป็นการเช็คระดับ video สอดคล้องกับความต้องการระบบที่ว่า

“ภาพวิดีโอต้องมีความถี่ไม่น้อยกว่า 30 fps (เฟรมต่อวินาที) ขนาดของวิดีโอความละเอียดอย่างน้อย full HD และวิดีโอบันทึกภาพการเดินไปและกลับต้องมีความยาววิดีโอไม่น้อยกว่า 15 วินาที ทั้ง 2 กล้อง”

“รูปแบบการบันทึกชื่อไฟล์ 001_walk_F.mp4, 001_walk_S.mp4, โดย 001 หมายถึง รหัสแทน
 อาสาสมัคร, walk หมายถึง ชีวมิติท่าทางเดิน, F หมายถึง เดินด้านหน้า, S หมายถึง เดินด้านข้าง”
 ซึ่ง full HD คือ ความละเอียดของภาพ 1920 x 1080 pixel

การเช็คทั้งสี่รายการนี้ใช้ฟังก์ชันเดียวกันกับการตรวจสอบข้อมูลใบหน้า (qc/checks/check_metadata.py)
 ซึ่งได้อธิบายรายละเอียดไว้แล้วในหัวข้อ 5.2.1 ต่างกันเพียงค่าเกณฑ์ที่ใช้ดังนี้

การเช็ค	เกณฑ์ของใบหน้า	เกณฑ์ของการเดิน
check_container	.mp4 และเป็นภาพสี RGB	.mp4 และเป็นภาพสี RGB
check_fps	≥ 5 fps	≥ 30 fps
check_duration	≥ 40 วินาที	≥ 15 วินาที
check_resolution	≥ 180 × 180	≥ 1920 × 1080

โดยสามารถปรับเกณฑ์ได้ใน config.yml

check_body_height

ไฟล์ qc/checks/check_body_height.py

```
def check_body_height(
    landmarks_norm,
    *,
    min_ratio: float = 0.5,
    min_visibility: float = _DEFAULT_MIN_VISIBILITY,
    min_visible_landmarks: int = _MIN_VISIBLE_LANDMARKS,
):
    if not landmarks_norm:
        return (False, "No pose landmarks provided")

    ys = []
    for lm in landmarks_norm:
        vis = getattr(lm, "visibility", None)
        if vis is None or vis >= min_visibility:
            ys.append(float(lm.y))

    if len(ys) < min_visible_landmarks:
        return (
            False,
            f"body_height=0.00; only {len(ys)} visible landmark(s) "
            f"< {min_visible_landmarks} needed to measure height",
        )

    ratio = max(ys) - min(ys)

    if ratio >= min_ratio:
        return (True, f"body_height={ratio:.2f} >= {min_ratio:.2f}")
    return (False, f"body_height={ratio:.2f} < {min_ratio:.2f}")
```

สอดคล้องกับความต้องการระบบที่ว่า "ตัวคนต้องมีความสูงอย่างน้อยครึ่งหนึ่งของความสูงภาพ" โดยการเช็คนี้นี้จะเช็คความสูงของบุคคลในภาพมีขนาดอย่างน้อยครึ่งหนึ่งของความสูงภาพหรือไม่ โดยวัดจากระยะในแนวตั้ง (แกน y) ระหว่าง landmark ที่อยู่สูงที่สุดกับ landmark ที่อยู่ต่ำที่สุด เทียบกับความสูงของเฟรม และจะวัดเฉพาะเฟรมที่ 0 ที่ดึงมาจากวิดีโอเท่านั้น เพราะเฟรมแรกเป็นจุดที่อาสาสมัครอยู่ห่างจากกล้องมากที่สุด จึงปรากฏในภาพมีขนาดเล็กที่สุด จึงเสมือนว่าเป็นการเช็คระดับ video เพราะเช็คครั้งเดียวต่อทั้งวิดีโอ

5.4.2 การเช็คระดับ frame

check_person_detected, check_single_person

ไฟล์ `qc/pipelines/walk.py` ซึ่งมีการเรียกใช้ฟังก์ชัน `create_pose_landmarker`, `detect_pose` จาก `qc/checks/pose_landmarker.py`

```
p = detect_pose(sf.image, detector=detector,
                input_color_space=sf.color_space,
                min_real_visibility=min_real_vis,
                min_real_landmarks=min_real_lms)

multiple_persons = (not p.ok) and \
    str(p.message).startswith("Multiple persons detected")

if p.ok:
    pd_status = "PASS"
    pd_reason = f"frame={sf.frame_index} person detected"
else:
    pd_status = "FAIL"
    pd_reason = f"frame={sf.frame_index} {p.message}"
add("check_person_detected", pd_status, pd_reason,
    frame_index=sf.frame_index, level="frame")
```

ฟังก์ชัน `detect_pose` จะทำการตรวจจับบุคคลจากการเรียกใช้โมเดล MediaPipe Pose Landmarker ผ่าน

การใช้ Tasks API และโปรแกรมจะเช็คผลที่ได้รับกลับมาผ่านทาง instance type `PoseResult`

```
@dataclass
class PoseResult:
    ok: bool
    message: str
    landmarks_px: Optional[list] = None          # [(x, y, z)] pixel space
    landmarks_norm: Optional[Any] = None         # raw normalized landmark list
    bbox: Optional[tuple] = None                 # (x, y, w, h) px, 10% margin
    norm_box: Optional[tuple] = None             # (x, y, w, h) normalized
    visibility: Optional[list] = None            # per-landmark visibility [0,1]
    world_landmarks: Optional[Any] = None       # raw world landmark list
```

สิ่งที่ `PoseResult` มีเพิ่มเติมจาก `FaceResult` และ `HandResult` คือ `visibility` ซึ่งเป็นคะแนนความ

น่าเชื่อถือของ landmark แต่ละจุด ในช่วง 0 ถึง 1 คำนี้นี้ทำให้สามารถแยกแยะได้ว่า landmark ใดที่โมเดลตรวจจับพบจริงในเฟรม และ landmark ใดที่โมเดลเพียงคาดเดาตำแหน่งไว้นอกเฟรม ซึ่งเป็นข้อมูลเพื่อการตรวจจับใบหน้าและฝ่ามือไม่มีให้

การเช็คนี้เป็นการเช็คลำดับแรกของระดับ frame ซึ่งการเช็คระดับ frame รายการอื่นต้องอาศัยผลลัพธ์ PASS ของการเช็คนี้ โดยจำเป็นต้องมีการแยกกรณีไม่พบบุคคลออกจากกรณีพบหลายบุคคล เนื่องจากทั้งสองกรณีทำให้ p.ok เป็น FALSE เหมือนกัน แต่มีความหมายต่างกันโดยสิ้นเชิง จึงถูกแยกรายงานใน CSV report เป็นคนละการเช็ค มีรายละเอียดคือ หากไม่พบบุคคลในเฟรม รายงานเป็น check_person_detected สถานะ FAIL ในระดับ frame ซึ่งจะถูกรวบรวมผลด้วยว่า FAIL ก็ต่อเมื่อมี frames ที่ FAIL มากกว่าร้อยละ 20 แต่ถ้าหากพบบุคคลมากกว่าหนึ่งคน รายงานเป็น check_single_person สถานะ FAIL ในระดับ video ซึ่งจะ ทำให้ทั้งวิดีโอ FAIL ทั้งนี้แม้จะพบเพียงเฟรมเดียวก็ตามเนื่องจากข้อกำหนดระบุให้ถ่ายทำอาสาสมัครเพียงคนเดียว

check_person_fully

ไฟล์ qc/checks/check_person_fully.py

```
_REQUIRED = (
    (2, "left_eye"), (5, "right_eye"),
    (7, "left_ear"), (8, "right_ear"),
    (19, "left_hand"), (20, "right_hand"),
    (31, "left_foot"), (32, "right_foot"),
)

def check_person_fully(landmarks_px, image_width, image_height,
                       *, margin_px: int = 0):
    if not landmarks_px:
        return (False, "No pose landmarks provided")

    n = len(landmarks_px)
    lo_x = margin_px
    hi_x = image_width - margin_px
    lo_y = margin_px
    hi_y = image_height - margin_px

    missing = []
    out_of_frame = []
    for idx, name in _REQUIRED:
        if idx >= n:
            missing.append(name)
            continue
        x, y = landmarks_px[idx][0], landmarks_px[idx][1]
        if x < lo_x or x > hi_x or y < lo_y or y > hi_y:
            out_of_frame.append(name)

    if missing or out_of_frame:
        parts = []
        if missing:
            parts.append("missing: " + ", ".join(missing))
        if out_of_frame:
            parts.append("out of frame: " + ", ".join(out_of_frame))
        return (False, "person not fully visible (" + "; ".join(parts) + ")")

    return (True, "person fully visible (all 8 key landmarks in frame)")
```

เชื่อว่าเห็นร่างกายของอาสาสมัครอยู่ในเฟรมทั้งตัวหรือไม่ โดยใช้ landmark 8 จุด ได้แก่ 2 (ตาซ้าย) 5 (ตาขวา) 7 (หูซ้าย) 8 (หูขวา) 19 (นิ้วชี้มือซ้าย) 20 (นิ้วชี้มือขวา) 31 (ปลายเท้าซ้าย) 32 (ปลายเท้าขวา) หากจุดใดจุดหนึ่งหายไปหรืออยู่นอกกรอบภาพจะคืนค่ากลับเป็น FALSE พร้อมระบุว่าส่วนใดของร่างกายที่ถูกตัดออกจากเฟรม แต่หากพบทุกจุดคืนค่า TRUE

check_person_blur

ไฟล์ [qc/checks/check_face_blur.py](#)

เช็คความคมชัดของภาพบริเวณร่างกาย โดยใช้ฟังก์ชันเดียวกันกับการเช็คความคมชัดของใบหน้า (check_face_blur) ซึ่งได้อธิบายรายละเอียดการทำงานไว้แล้วในหัวข้อ 5.2.2 โดยความต่างคือการใช้ bounding box ของร่างกายที่ได้จาก detect_pose และถ้าหากในกรณีที่ภาพมืดเกินไปหรือมีคอนทราสต์ต่ำเกินไปจนไม่สามารถประเมินความคมชัดได้อย่างน่าเชื่อถือ ฟังก์ชันจะคืนค่ากลับเป็น None ซึ่งจะถูกละผลเป็นสถานะ SKIP เช่นเดียวกับการตรวจสอบใบหน้า

check_occlusion

ไฟล์ [qc/checks/check_occlusion_yolo.py](#), [qc/checks/yolo_detector.py](#)

สอดคล้องกับความต้องการระบบที่ว่า "...และเป็นสถานที่ปิด สภาพพื้นที่ต้องปลอดภัย ไม่มีสิ่งกีดขวาง" เป็นการเช็คเดียวที่ใช้โมเดลนอกตระกูล MediaPipe โดยใช้โมเดล YOLOv8-nano บนชุดข้อมูล COCO ซึ่งสามารถตรวจจับวัตถุได้ 80 classes โดย class ที่ 0 คือ person หรือคน ส่วนอีก 79 classes เป็นวัตถุอื่นๆที่ไม่ใช่คน เช่น จักรยาน รถยนต์ แก้วอี้ กระเป๋าเป้ โทรศัพท์มือถือ กระดาษต้นไม้ ที่นอน เป็นต้น

หลักการทำงานคือ วัตถุที่ตรวจจับได้ซึ่งไม่ใช่ประเภท person ถือเป็นสิ่งแปลกปลอมที่อาจเป็นสิ่งกีดขวาง แล้วจึงเชื่อว่าวัตถุนั้นกีดขวางทางเดินจริงหรือไม่ ซึ่งใช้เกณฑ์ต่างกันระหว่างกล้องหน้าและกล้องด้านข้าง

กล่องที่ 1 (_F) ใช้เกณฑ์การซ้อนทับของกรอบ

```
def _boxes_overlap(a, b) -> bool:
    ax, ay, aw, ah = a
    bx, by, bw, bh = b
    ix = max(ax, bx)
    iy = max(ay, by)
    ix2 = min(ax + aw, bx + bw)
    iy2 = min(ay + ah, by + bh)
    return (ix2 - ix) > 0 and (iy2 - iy) > 0
```

วัตถุจะถือเป็นสิ่งกีดขวางก็ต่อเมื่อกรอบ bbox ของวัตถุ ซ้อนทับกับกรอบ bbox ของอาสาสมัคร หากวัตถุอยู่ด้านข้างโดยไม่ซ้อนทับจะไม่ถือเป็นสิ่งกีดขวาง หากมีสิ่งกีดขวางจะรายงานผล frame นั้นเป็น FAIL

กล้องที่ 2 (_S) ใช้เกณฑ์แนวระดับเท้า

```
def _in_foot_band(obj_bbox, foot_line_y: float, person_h: float,
                  ratio: float) -> bool:
    band_half = ratio * person_h
    obj_bottom = obj_bbox[1] + obj_bbox[3]
    return (foot_line_y - band_half) <= obj_bottom <= (foot_line_y + band_half)
```

สำหรับกล้องด้านข้าง เกณฑ์การซ้อนทับของกรอบใช้วัดไม่ได้ เพราะวัตถุอาจวางอยู่แนวหลังเป็น

background แต่ไม่ได้ขวางทางเดิน ทำให้วัตถุที่อยู่ห่างออกไปด้านหลังอาสาสมัครปรากฏเป็นกรอบที่ซ้อนทับกับกรอบของอาสาสมัครในเฟรมได้ ระบบจึงเปลี่ยนไปวัดแนวระดับเท้า โดยอาศัยหลักการว่าสิ่งกีดขวางที่วางอยู่บนพื้นจริงมีขอบล่างของวัตถุอยู่ในระดับเดียวกับเท้าที่เหยียบพื้นของอาสาสมัคร แนวระดับเท้าคำนวณจากค่า y ที่มากที่สุดของ landmark ปลายเท้าทั้งสองข้าง (จุดที่ 31 และ 32) เนื่องจากในระบบพิกัดของภาพ ค่า y เพิ่มขึ้นเมื่อเลื่อนลงด้านล่าง ค่า y ที่มากที่สุดจึงเป็นเท้าที่อยู่ต่ำกว่า แแถบที่กว้างร้อยละ 10 ของความสูงตัวคนในแต่ละด้านของแนวระดับเท้า (ทั้งด้านที่เหนือขึ้นไป และด้านที่ต่ำลงมา) รวมเป็นแถบกว้างร้อยละ 20 หากมีขอบล่างของสิ่งกีดขวางอยู่ในแนวแถบกว้างตามแกน y นี้จะถือว่าสิ่งกีดขวางในเฟรมนั้นและรายงานผลเป็น FAIL

config.yml

walk:

occlusion:

enabled: true

conf: 0.6

frame_fail_ratio: 0.2

overlay_draw: all # "all" | "overlap"

foot_band_ratio: 0.10

ค่า conf คือระดับความมั่นใจขั้นต่ำที่วัตถุจะถูกนับเป็นสิ่งแปลกปลอม การกำหนดค่านี้จำเป็นเพื่อป้องกันไม่ให้ตรวจจับผิดพลาดด้วยความมั่นใจต่ำ เช่น จากการทดลองบนข้อมูลทดสอบพบว่าการตรวจจับพบรถยนต์บนบริเวณที่ควรจะเป็นหน้าต่างด้วยความมั่นใจเพียง 0.38 ส่วนการปรับ overlay_draw ควบคุมเฉพาะการ

แสดงผลบน overlay video เท่านั้น ไม่มีผลต่อการตัดสิน PASS FAIL โดยค่า all จะวาดกรอบของวัตถุทุกชิ้นที่ตรวจจับได้ เพื่อให้ผู้ตรวจสอบเห็นว่าโมเดลตรวจจับอะไรได้บ้าง ส่วนค่า overlap จะวาดเฉพาะวัตถุที่ถือเป็นสิ่งกีดขวางตามเกณฑ์ที่ใช้อยู่

ในกรณีที่ไม่สามารถระบุตำแหน่งของอาสาสมัครในเฟรมนั้นได้เลย ทั้งจาก MediaPipe และจาก YOLO (สำหรับ YOLO คือไม่สามารถ detect class 0 person ได้) หากพบสิ่งแปลกปลอมในเฟรมนั้นจะถือว่าไม่ผ่านหรือ FAIL เนื่องจากไม่มีข้อมูลเพียงพอที่จะยืนยันได้ว่าวัตถุนั้นไม่ได้กีดขวางทางเดิน

5.4.3 การเช็คระดับ sequence

check_walk_direction

ไฟล์ qc/check_walk_direction ฟังก์ชัน _polarity_series

สอดคล้องกับความต้องการระบบที่ว่า "บันทึกภาพวิดีโอโดยเดินเข้าหากล้องและเดินหันหลังออกจากกล้อง (กล้องที่ 1) และบันทึกภาพวิดีโอเดินผ่านกล้องไปด้านซ้ายและขวา (กล้องที่ 2) โดยกล้องหนึ่งและสองต้องบันทึกภาพพร้อมกัน" การเช็คนี้ตรวจสอบว่าอาสาสมัครเดินตามทิศทางและตามลำดับที่กำหนดหรือไม่ตลอดทั้งวิดีโอ ซึ่งปัญหาคือโมเดลตรวจจับ pose รายงานเพียงตำแหน่ง landmark ในแต่ละเฟรมเท่านั้น ไม่ได้รายงานทิศทางของบุคคลในวิดีโอ จึงไม่สามารถใช้ค่าที่ได้จากโมเดลและต้องมาคำนวณเอาเอง

มุมกล้อง _F (เข้าหากล้องและหันหลังออกจากกล้อง) ใช้ body_scale (ความสูงกรอบ ÷ ความสูงภาพ) ใช้หลักการที่ว่าเดินเข้าหากล้องทำให้ตัวใหญ่ขึ้น เดินออกจากกล้องทำให้ตัวเล็กลง

มุมกล้อง _S (ซ้ายและขวา) ใช้ centroid_x ใช้หลักการที่ว่าเดินไปทางซ้ายทำให้ค่าลดลง เดินไปทางขวาทำให้ค่าเพิ่มขึ้น

```
def _polarity_series(values, *, smooth_window, eps):
    """Smoothed step-to-step polarity: +1 rising, -1 falling, 0 flat."""
    sm = _moving_average(values, smooth_window)
    signs = []
    for a, b in zip(sm, sm[1:]):
        d = b - a
        if d > eps:
            signs.append(+1)
        elif d < -eps:
            signs.append(-1)
        else:
            signs.append(0)
    return signs
```

ดึงข้อมูลจาก timeline โดยตัดเฟรมที่ตรวจจับ pose ไม่ได้ ออก เฉลี่ยแบบ moving average หาทิศทาง การเปลี่ยนแปลง ในแต่ละช่วง โดยกำหนดเป็น +1 (เพิ่มขึ้น), -1 (ลดลง), หรือ 0 (คงที่) โดยการเปลี่ยนแปลงที่ น้อยกว่าค่า eps จะถือว่าคงที่หรือแปลว่ากำลังหยุดเดินอยู่ หลังจากนั้นรวมเป็นช่วงการเคลื่อนไหว โดยช่วงที่ มีทิศทางเดียวกันติดต่อกันตั้งแต่ min_run เฟรมขึ้นไปจึงจะนับเป็นหนึ่งช่วง

```
class PhaseSequenceFSM:
    def __init__(self, required):
        self.required = tuple(required)
        self.n = len(self.required)
        self.state = 0
        self.consumed = []

    @property
    def accepting(self):
        return self.state >= self.n

    def feed(self, phase):
        self.consumed.append(phase)
        if self.accepting:
            return self.state # trailing phase: ignore, stay accepted
        if phase == self.required[self.state]:
            self.state += 1
        # else: unexpected phase -> no transition (machine stalls on this step)
        return self.state

    def run(self, phases):
        for p in phases:
            self.feed(p)
        return self.accepting
```

หลังจากนั้นแปลผลลำดับช่วงของการเคลื่อนไหวที่ได้ว่าเป็น PASS FAIL ด้วย Finite State Machine ซึ่งจะ PASS ต่อเมื่อช่วงการเคลื่อนไหวเกิดขึ้นตามลำดับที่กำหนดเท่านั้น โดยสถานะของ FSM หมายถึงจำนวนช่วงที่ ตรงตามลำดับที่ผ่านมาแล้ว ซึ่งการใช้ FSM ทำให้สามารถแยกแยะกรณีเดินสลับลำดับได้ เช่น ในวิดีโอ _F หากอาสาสมัครเดินออกจากกล้องก่อนแล้วจึงเดินเข้าหากล้อง ลำดับไม่ตรงตามข้อกำหนด จะได้สถานะ FAIL

ตัวอย่างผลลัพธ์การรัน walk_002_F_result.csv

#	volunteer_id	data_type	filename	check_level	check_name	final_status	# total	# pass	# fail	# review	# skip	reason
Missing:	0 (0%)	Missing:	0 (0%)	Missing:	0 (0%)	Missing:	0 (0%)	Missing:	0 (0%)	Missing:	0 (0%)	Missing:
Distinct:	1 (8%)	Distinct:	1 (8%)	Distinct:	3 (25%)	Distinct:	12 (100%)	Distinct:	2 (17%)	Distinct:	33%	Distinct:
			100%	100%	video	58%	check_container	8%	PASS	83%		frame=636 Multiple persons detected: 2 (ju...
					frame	33%	check_fps	8%	FAIL	17%		container ok (mp4, 3 channels, codec=h264)
					sequence	8%	check_duration	75%				fps=30.00 >= 30
					Other							Other
0	2	walk_F	002_walk_F.mp4	video	check_container	PASS	1	1	0	0	0	container ok (mp4, 3 channels, codec=h264)
1	2	walk_F	002_walk_F.mp4	video	check_fps	PASS	1	1	0	0	0	fps=30.00 >= 30
2	2	walk_F	002_walk_F.mp4	video	check_duration	PASS	1	1	0	0	0	duration=25.0 >= 15
3	2	walk_F	002_walk_F.mp4	video	check_resolution	PASS	1	1	0	0	0	resolution=1920x1080 >= 1920x1080
4	2	walk_F	002_walk_F.mp4	video	frames_sampled	PASS	1	1	0	0	0	sampled 126 frames
5	2	walk_F	002_walk_F.mp4	frame	check_person_detected	PASS	126	120	6	0	0	frame=636 Multiple persons detected: 2 (judged fail-ra
6	2	walk_F	002_walk_F.mp4	video	check_single_person	FAIL	1	0	1	0	0	frame=636 Multiple persons in frame: Multiple pers
7	2	walk_F	002_walk_F.mp4	frame	check_person_fully	PASS	126	120	6	0	0	frame=636 Multiple persons detected: 2 (judged fail-ra
8	2	walk_F	002_walk_F.mp4	video	check_body_height	FAIL	1	0	1	0	0	frame=0 body_height=0.27 < 0.50
9	2	walk_F	002_walk_F.mp4	frame	check_person_blur	PASS	126	120	6	0	0	frame=636 Multiple persons detected: 2 (judged fail-ra
10	2	walk_F	002_walk_F.mp4	frame	check_occlusion	PASS	126	118	8	0	0	frame=672 obstructing objects: bench (0.67) (judged fi
11	2	walk_F	002_walk_F.mp4	sequence	check_walk_direction	PASS	1	1	0	0	0	F: forward-then-away OK (detected: approach (growing

6. การตั้งค่า configuration ของโปรแกรม

ไฟล์ : config.yml

ส่วน path

```
paths:
  input_root: "data"
  report_dir: "reports"
  report_csv: "reports/qc_report.csv"
  report_json: "reports/qc_report.json"
```

แสดง path ของ input และ output

ส่วน filenames

```
filenames:
  id_width_policy: "any" # any | fixed_3 | fixed_4
  required:
    face_rgb: "(?P<volunteer_id>\\d+)_face_rgb\\.mp4$"
    face_depth: "(?P<volunteer_id>\\d+)_face_depth\\.mp4$"
    face_ir1: "(?P<volunteer_id>\\d+)_face_ir1\\.mp4$"
    face_ir2: "(?P<volunteer_id>\\d+)_face_ir2\\.mp4$"
    face_thermal: "(?P<volunteer_id>\\d+)_face_thermal\\.mp4$"
    palm_L_N: "(?P<volunteer_id>\\d+)_palm_L_N\\.jpg$"
    palm_L_RL: "(?P<volunteer_id>\\d+)_palm_L_RL\\.jpg$"
    palm_L_RR: "(?P<volunteer_id>\\d+)_palm_L_RR\\.jpg$"
    palm_L_PU: "(?P<volunteer_id>\\d+)_palm_L_PU\\.jpg$"
    palm_L_PD: "(?P<volunteer_id>\\d+)_palm_L_PD\\.jpg$"
    palm_R_N: "(?P<volunteer_id>\\d+)_palm_R_N\\.jpg$"
    palm_R_RL: "(?P<volunteer_id>\\d+)_palm_R_RL\\.jpg$"
    palm_R_RR: "(?P<volunteer_id>\\d+)_palm_R_RR\\.jpg$"
    palm_R_PU: "(?P<volunteer_id>\\d+)_palm_R_PU\\.jpg$"
    palm_R_PD: "(?P<volunteer_id>\\d+)_palm_R_PD\\.jpg$"
    walk_F: "(?P<volunteer_id>\\d+)_walk_F\\.mp4$"
    walk_S: "(?P<volunteer_id>\\d+)_walk_S\\.mp4$"
```

สามารถปรับ id_with_policy: any เพื่อให้รับ volunteer_id ที่ไม่จำกัดขนาดความยาวสตริง, fixed_3 เพื่อรับ volunteer_id ที่มีความยาวสตริง 3 เท่านั้น (เช่น "001"), fixed_4 เพื่อรับ volunteer_id ที่มีความยาวสตริง 4 เท่านั้น (เช่น "0001") โดยมีการตั้งการจำกัดความยาวสตริงเนื่องจากในเอกสารความต้องการระบบมีการแสดงตัวอย่างชื่อไฟล์ทั้งแบบที่เป็นสตริงความยาว 3 (001_face_rgb.mp4) และแบบที่เป็นสตริงความยาว 4 (0001_palm_L_N.jpg) และส่วน required แสดง RegEx string ที่นำไปใช้ในการตรวจสอบใน validate_filenames.py pipeline

ส่วน video

```
video:
  max_frames: 6000
```

สามารถกำหนดจำนวน frames สูงสุดจาก video ได้เพื่อป้องกันมิให้รันในกรณีที่ต้องประมวลผล frames จำนวนมากเกินไป

ส่วน models

```
models:
  face_landmarker:
    model_path: "models/face_landmarker.task"
    num_faces: 10
    min_face_detection_confidence: 0.6
    min_face_presence_confidence: 0.5
    min_tracking_confidence: 0.5

  face_detection:
    model_selection: 1
    min_detection_confidence: 0.5

  hand_landmarker:
    model_path: "models/hand_landmarker.task"

  hands:
    max_num_hands: 1
    min_detection_confidence: 0.1
    multiple_hands_policy: "FAIL"

  pose_landmarker:
    model_path: "models/pose_landmarker.task"
    num_poses: 10
    min_pose_detection_confidence: 0.5
    min_pose_presence_confidence: 0.5
    min_tracking_confidence: 0.5
    min_real_visibility: 0.5
    min_real_landmarks: 8

  yolo:
    model_path: models/yolov8n.pt
    conf: 0.6
```

MediaPipe FaceMesh สามารถตรวจจับได้มากที่สุด 10 ใบหน้า และ Pose สามารถตรวจจับบุคคลได้มากที่สุด 10 คน ในแต่ละตระกูล face, hand, หรือ pose สามารถปรับ detection confidence ได้ ซึ่งเป็นระดับความมั่นใจในการตรวจจับเบื้องต้นของใบหน้า มือ หรือท่าทางการเดิน เพื่อค้นหาว่าอยู่บริเวณใดของภาพ นอกจากนี้ในแต่ละตระกูล face, หรือ pose สามารถปรับ presence confidence ได้ ซึ่งเป็นระดับความมั่นใจในการตรวจจับเจอ landmark ย่อยแต่ละจุดที่อยู่ในขอบเขตของเฟรม นอกจากนี้ในแต่ละตระกูล face, หรือ pose สามารถปรับ min_tracking_confidence มีความสำคัญในกรณีที่ปรับ running_mode เป็น VIDEO หรือ LIVE_STREAM ในการกำหนดว่าโมเดลจะใช้ข้อมูลจากเฟรมที่ได้ก่อนหน้ามาประกอบหรือไม่หรือจะตรวจจับใหม่ทั้งเฟรม

ส่วน face

face:

```
extension: ".mp4"  
container: "MPEG-4"
```

...

ค่าเกณฑ์ของการตรวจสอบข้อมูลใบหน้า มีค่าที่สำคัญคือ

blur.preprocess กำหนดวิธีการปรับภาพก่อนวัดความคมชัด โดยมีตัวเลือก 4 แบบ ได้แก่ none (ไม่ปรับ), clahe (ค่าเริ่มต้น), lideg และ lidel การเปลี่ยนค่านี้จะ เปลี่ยนคะแนนความคมชัดที่วัดได้ จึงต้องปรับค่า threshold ตามไปด้วย

blur.min_luma และ blur.min_contrast เป็นเกณฑ์ที่กำหนดว่าเมื่อใดจึงจะ SKIP ข้ามการ check ความคมชัด เพราะถ้าหากภาพมืดเกินไปหรือมีคอนทราสต์ต่ำเกินไป การวัดความคมชัดจะไม่น่าเชื่อถือ ระบบจึงให้สถานะ SKIP แทนการตัดสิน FAIL และปล่อยให้การ check_brightness ทำงานแทน

eyes_open.blink_threshold ถูกกำกับไว้ว่า [CONFIRM] โดยค่า 0.5 เป็นจุดกึ่งกลาง ตามธรรมชาติของคะแนน blendshape ที่มีช่วง 0 ถึง 1 ทั้งนี้จากการสังเกตพบว่าโมเดลมักให้คะแนนตาที่ปิดชัดเจนใกล้ 0.9 และตาที่เปิดใกล้ 0.0 ถึง 0.1

ส่วน turn_sequence เป็นส่วนที่กำหนดลำดับการหันหน้าให้ผ่านได้ และระยะเวลาที่ต้องค้างในแต่ละท่า

turn_sequence:

```
enabled: true  
total_min_duration_sec: 40  
  
# "exact": observed sequence must equal an accepted order exactly.  
# "contiguous_subsequence": accepted order may appear as one contiguous  
# window inside the observed sequence; prefix/suffix noise is ignored.  
# Use exact for final/spec QC; use contiguous_subsequence for legacy data  
# where volunteers may have turned before pressing/after stopping record.  
sequence_match_mode: "exact"  
order_policy: "FAIL" # no human-review capacity at 1,500 scale;  
# a turn-order mismatch is a non-conforming file -> FAIL.  
short_hold_policy: "FAIL" # a turn held < its minimum duration did not  
# meet the spec's hold time -> FAIL.  
unconfirmed_gap_policy: "FAIL" # a no-face gap that we cannot positively  
# confirm as a real turn.  
  
accepted_orders:  
  spec:  
    - "front"  
    - "left"  
    - "front"  
    - "right"
```



```

- "front"
- "down"
- "front"
- "up"
- "front"
audio_prompt:
- "front"
- "left"
- "front"
- "right"
- "front"
- "up"
- "front"
- "down"
- "front"

hold_seconds:
  front_initial: 5
  front_between_movements: 2
  left: 2
  right: 2
  down: 2
  up: 2

tolerance:
  side_yaw_tolerance_deg: 30
  tilt_pitch_tolerance_deg: 20
  min_detected_step_ratio: 0.6
  front_zone_yaw_deg: 15
  front_zone_pitch_deg: 15
  label_jitter_tolerance_sec: 0.15

```

accepted_orders มีสองลำดับที่ยอมรับได้ เนื่องจากลำดับที่ระบุในเอกสารข้อกำหนด (spec) และ ลำดับที่เสียงบรรยายในการเก็บข้อมูลจริงสั่ง (audio_prompt) มีความแตกต่างกัน ในลำดับของท่าก้มและท่าเงย

sequence_match_mode กำหนดความเข้มงวดในการเทียบลำดับ โดย exact หมายถึง ลำดับที่ตรงพบต้องตรงกับลำดับที่กำหนดไว้ได้ทุกประการ ไม่สามารถมีลำดับเกินหรือขาดได้ เช่น down => front => left => front => right => front => up => front => down => front จะถือว่าไม่ผ่านเพราะมีส่วน down นำหน้า ส่วน contiguous_subsequence ยอมให้มีการเคลื่อนไหวก่อนหรือหลังลำดับหลักได้ ซึ่งเหมาะกับข้อมูลที่อาสาสมัครอาจขยับก่อนกดบันทึกหรือหลังหยุดบันทึกวิดีโอ

unconfirmed_gap_policy เป็นกรณีที่ระบบไม่สามารถประเมินใบหน้าได้ว่าช่วงที่ตรวจไม่พบใบหน้านั้นเป็นการหันหน้าจริงหรือไม่ (ช่วงการหันหน้าที่สามารถคาดเดาได้ว่าอาสาสมัครกำลังหันหน้าอยู่จะต้องมี front นำหน้าและ front ตามหลังทันทีเท่านั้น กรณีที่ไม่แน่ใจว่าเป็นการหันหน้าจริงหรือไม่ เช่น front => left => down => front) การตั้งค่าเป็น FAIL หมายความว่าไฟล์ที่มีช่วงเช่นนี้จะถูกปฏิเสธทันที

ส่วน palm

```
palm:
  extension: ".jpg"
  pattern_keys:
```

ค่าเกณฑ์ของการตรวจสอบข้อมูลฝ่ามือ มีค่าที่มีความสำคัญคือ

brightness.dark_threshold และ bright_threshold สามารถปรับได้เพื่อปรับค่า threshold ในการ check_brightness มีลักษณะคล้ายกับการเช็คค่าความสว่างของใบหน้า แต่ฝ่ามือภายใต้อุปกรณ์ถ่ายภาพ โดยเฉพาะทางอาจให้ค่าความสว่างต่างออกไปจึงต้องแยก threshold ออกมาเป็นของ palm เองโดยเฉพาะ spread ทุกค่า ควรปรับเทียบกับภาพที่กางนิ้วจริงและภาพที่นิ้วชิดกันจริง

```
spread:
  enabled: true
  eval_poses: ["N"]
  min_gap_inter_tip_deg: 2.0      # index-middle/middle-ring/ring-pinky (tip source)
  min_gap_thumb_tip_deg: 15.0     # thumb-index (tip source)
  min_gap_inter_pip_deg: 2.0      # inter-finger (pip source; PIPs fan less)
  min_gap_thumb_pip_deg: 15.0     # thumb-index (pip source)
```

hand_pose.spread สามารถปรับค่าองศาของแต่ละคุ่มที่จะนำไปใช้ในการตรวจสอบว่าอาสามัครกางนิ้วมือออกแล้วหรือไม่

```
angle:
  max_abs_roll_deg: 45
  max_abs_pitch_deg: 45
  check_angle_enabled: true
  max_abs_deg: 45
  min_rotation_deg: 10           # a deliberate pose must rotate >= this
  neutral_tol_deg: 15           # used by the per-image absolute path
(check_palm_pose)
  n_reference_max_deg: 15       # N must read |roll|, |pitch| <= this to be a trusted
baseline; calibrate
  off_axis_tol_deg: 45          # a rotated pose's OTHER axis must stay within +/-
this
  # HAND SIGN CALIBRATION
  hand_sign_overrides: {}
```

angle.min_rotation_deg และ neutral_tol_deg เป็นค่าที่กำหนดขึ้นเองว่าการหมุนด้วยองศาขั้นต่ำที่ถือว่าผ่านมีค่าองศาเท่าไร

hand_sign_overrides ใช้ปรับเครื่องหมายของมูมรายท่าในกรณีที่อุปกรณ์ถ่ายภาพมีการกลับภาพ ซึ่งจะทำให้ทิศทางการเอียงที่วัดได้กลับด้าน โดยปกติไม่ต้องตั้งค่านี้นี้ แต่หากพบว่า ภาพท่า RL ถูกตัดสินว่าเอียงผิดทิศ ทั้งที่ได้ถ่ายรูปฝ่ามือมาอย่างถูกต้องแล้วควรตรวจสอบค่านี้นี้

ส่วน walk

ค่าเกณฑ์ของการตรวจสอบข้อมูลท่าทางการเดิน

```
walk:
  extension: ".mp4"
  pattern_keys:
    - "walk_F"
    - "walk_S"
  ...
```

ค่าที่มีความสำคัญ เช่น

brightness.enabled: false ปัจจุบันไม่ใช้การเช็คความสว่างของท่าทางการเดินในแต่ละ frame

```
occlusion:
  enabled: true
  conf: 0.6
  frame_fail_ratio: 0.2
  overlay_draw: all
  foot_band_ratio: 0.10
```

```
direction:
  smooth_window: 5          # moving-average window over the frame series
  min_run: 3                # consecutive frames of one polarity = a phase
  eps: 0.002                # |delta| below this per step is treated as flat
  min_frames: 8             # fewer usable frames -> SKIP (too short)
```

occlusion.conf ค่านี้นี้ตั้งไว้ที่ 0.6 ซึ่งค่อนข้างสูง หมายความว่าวัตถุจะถูกลบเป็น สิ่งกีดขวางก็ต่อเมื่อโมเดล

มั่นใจในการตรวจจับมาก การลดค่านี้นี้ลงจะทำให้ระบบตรวจจับสิ่งกีดขวางได้ไวขึ้น แต่มีความเสี่ยงที่โมเดลอาจตรวจจับสิ่งกีดขวางไม่ถูกต้อง

occlusion.foot_band_ratio เป็นค่าที่ใช้ปรับเปอร์เซ็นต์การซ้อนทับของแถบ occlusion ของมูมกลิ้งด้านข้าง

direction ปรับค่า threshold ได้

ส่วน report

```
report:
  aggregation:
    frame_fail_ratio: 0.2
    frame_review_ratio: 0.2
    per_check_fail_ratio:
      check_face_detected: 0.0
      check_single_person: 0.0
```

กำหนดการสรุปผลรวมจากการเช็คระดับเฟรม โดย frame_fail_ratio เป็นค่าเริ่มต้น ที่ใช้กับการเช็คระดับเฟรมทุกรายการ กล่าวคือ หากสัดส่วนเฟรมที่ไม่ผ่านหรือ FAIL มีมากกว่าร้อยละ 20 ของเฟรมที่ตัดสินใจได้ จะถือว่าไฟล์นั้นไม่ผ่านการเช็คในหัวข้อนั้น และเฟรมที่มีสถานะ SKIP จะไม่ถูกนับรวมในการคำนวณร้อยละนี้

ส่วน per_check_fail_ratio ใช้กำหนดค่าเฉพาะสำหรับการเช็คบางรายการที่ต้องการเกณฑ์ ต่างจากค่าเริ่มต้น โดยค่า 0.0 หมายความว่าเฟรมเดียวที่ไม่ผ่านจะทำให้ทั้งไฟล์ไม่ผ่านทันที ซึ่งใช้กับการเช็คที่ถือว่าเป็นข้อบกพร่องที่ไม่ควรเกิดขึ้น เช่น การตรวจพบใบหน้าหลายใบหน้าในเฟรม และการตรวจพบบุคคลหลายคนในวิดีโอท่าทางการเดิน และส่วนค่าเกณฑ์สัดส่วนของการตรวจสอบท่าทางการเดินถูกกำหนดไว้ในส่วน walk เรียบร้อยแล้ว

7. การใช้งานโปรแกรม

7.1 การใช้งานผ่าน command line

เป็นช่องทางที่ปรับแต่ง option ในการรันได้มากที่สุด และสามารถสั่งรันแยกเฉพาะบางส่วนของ pipeline เพื่อตรวจสอบเฉพาะข้อมูลบางประเภทได้ แต่ต้องอาศัยความเข้าใจในการรัน command และการกำหนด option

โปรแกรมมีไฟล์สำหรับสั่งรันทั้งหมด 4 ไฟล์ ดังนี้

ไฟล์	ขอบเขตการทำงาน
run_folder.py	สั่งรันทุกประเภทข้อมูลชีวมิติทั้งโฟลเดอร์
run_face.py	สั่งรันเฉพาะข้อมูลใบหน้า
run_palm.py	สั่งรันเฉพาะข้อมูลฝ่ามือ
run_walk.py	สั่งรันเฉพาะข้อมูลท่าทางการเดิน

โดยทั่วไปผู้ใช้งานควรใช้ run_folder.py เนื่องจากเป็นไฟล์ที่รันครบทุกขั้นตอนและตรวจสอบทั้งประเภทข้อมูลใบหน้า ฝ่ามือ และท่าทางการเดิน นอกจากนี้ยังสร้างไฟล์ผลลัพธ์รวม all_summary.csv ให้ ส่วนไฟล์ run_face.py, run_palm.py, และ run_walk.py เหมาะกับการทดสอบเฉพาะบางประเภทของข้อมูลหรือการตรวจสอบซ้ำเฉพาะไฟล์จากอาสาสมัครบางราย

7.1.1 run_folder.py

เป็นไฟล์สำหรับสั่งรันหลักของระบบ ทำหน้าที่สแกนโฟลเดอร์เพื่อหาไฟล์ของอาสาสมัคร แล้วส่งต่อให้ pipeline ของแต่ละประเภทข้อมูลชีวมิติ และเขียนไฟล์ผลลัพธ์รวมออกมา

รูปแบบการใช้งานพื้นฐาน

```
python run_folder.py <โฟลเดอร์ข้อมูล> [option ต่างๆ]
```

ตัวอย่างการใช้งาน

```
# รันตรวจสอบข้อมูลทุกประเภทในโฟลเดอร์ data/  
python run_folder.py data
```

รันเฉพาะข้อมูลฝ่ามือและท่าทางการเดิน (ข้ามใบหน้า)

```
python run_folder.py data --no-face
```

รันเฉพาะ 10 อาสาสมัครแรก เพื่อทดสอบก่อนรันจริงทั้งหมด

```
python run_folder.py data --limit 10
```

รันแบบเร็วที่สุด ไม่สร้างไฟล์ overlay และหยุดทันทีเมื่อพบข้อบกพร่อง

```
python run_folder.py data --no-overlay --fail-fast
```

ระบุโฟลเดอร์ผลลัพธ์เอง

```
python run_folder.py data --out-root reports_batch1
```

option ทั้งหมดของ run_folder.py

option สำหรับเลือกประเภทข้อมูลที่ต้องการตรวจสอบ

Option	ค่าเริ่มต้น	ความหมาย
--no-face	ตรวจสอบ	ข้ามการตรวจสอบข้อมูลใบหน้า
--no-palm	ตรวจสอบ	ข้ามการตรวจสอบข้อมูลฝ่ามือ
--no-walk	ตรวจสอบ	ข้ามการตรวจสอบข้อมูลท่าทางการเดิน
--no-filenames	ตรวจสอบ	ข้ามการตรวจสอบชื่อไฟล์และความครบถ้วน

ข้อมูลชีวมิติทุกประเภทจะถูกตรวจสอบโดยค่าเริ่มต้น การใช้ option เหล่านี้เป็นการละเว้นการตรวจสอบเฉพาะประเภทนั้น ซึ่งมีประโยชน์เมื่อต้องการตรวจสอบซ้ำเฉพาะบางประเภท โดยไม่ต้องเสียเวลารันประเภทอื่นๆ

option สำหรับควบคุมความเร็วและปริมาณการประมวลผล

Option	ค่าเริ่มต้น	ความหมาย
--limit N	ไม่จำกัด	ประมวลผลเฉพาะอาสาสมัคร N รายแรก
--sample-fps F	5.0	จำนวนเฟรมที่สุ่ม F เฟรมจากวิดีโอต่อหนึ่งวินาทีของวิดีโอต้นฉบับ
--no-overlay	สร้าง overlay	ไม่สร้างไฟล์ overlay สำหรับตรวจสอบด้วยสายตา

--fail-fast	ไม่หยุด	หยุดประมวลผลไฟล์นั้นทันทีเมื่อพบข้อบกพร่องเชิงโครงสร้าง
--no-detail	สร้างไฟล์ detail	ไม่สร้างไฟล์ผลลัพธ์ราย frame (*_detail.csv)

การใช้ --no-overlay ร่วมกับ --fail-fast เป็นการตั้งค่าที่เร็วที่สุด เหมาะกับการรันข้อมูลจำนวนมากเมื่อต้องการเพียงผลลัพธ์ PASS/FAIL โดยไม่ต้องการไฟล์สำหรับ debug หรือ overlay ทั้งนี้กลไก --fail-fast จะทำงานได้ก็ต่อเมื่อใช้ร่วมกับ --no-overlay เท่านั้น เนื่องจากการสร้าง overlay จำเป็นต้องอ่านเฟรมให้ครบทั้งวิดีโอ

ค่า --sample-fps มีผลโดยตรงต่อเวลาที่ใช้ในการประมวลผล การลดค่านี้อาจทำให้รันเร็วขึ้น แต่จะมีเฟรมที่ใช้ตัดสินน้อยลง ซึ่งอาจกระทบต่อความแม่นยำของการใช้ระดับ frame และ การเช็คทิศทางการเดินที่ต้องอาศัยลำดับการเคลื่อนไหวหากใส่ค่า 0 หรือค่าติดลบ ระบบจะประมวลผลทุกเฟรมของวิดีโอ

option สำหรับกำหนดที่อยู่ของไฟล์

Option	ค่าเริ่มต้น	ความหมาย
--config PATH	config.yml	ไฟล์ตั้งค่าที่ต้องการใช้
--out-root DIR	reports	โฟลเดอร์สำหรับเขียนผลลัพธ์
--all-summary PATH	<out-root>/all_summary	ที่อยู่ของไฟล์ผลลัพธ์รวม
--face-summary-csv PATH	<out-root>/face_summary.csv	ที่อยู่ของไฟล์สรุปผลใบหน้า
--palm-summary-csv PATH	<out-root>/palm_summary.csv	ที่อยู่ของไฟล์สรุปผลฝ่ามือ
--walk-summary-csv PATH	<out-root>/walk_summary.csv	ที่อยู่ของไฟล์สรุปผลท่าทางการเดิน
--append	เขียนทับ	เขียนต่อท้ายไฟล์ผลลัพธ์รวมเดิม แทนการเขียนทับ

option --append มีประโยชน์เมื่อต้องการแบ่งการรันข้อมูลจำนวนมากออกเป็นหลายรอบ โดยให้ผลลัพธ์ของทุกรอบรวมอยู่ในไฟล์ all_summary เดียวกัน

การใช้ --config ทำให้สามารถเตรียมไฟล์ตั้งค่าไว้หลายชุดสำหรับสถานการณ์ที่ต่างกัน เช่น ชุดหนึ่งสำหรับการตรวจสอบตามข้อกำหนดจริงทุกข้อ และอีกชุดหนึ่งที่มีการปรับค่า configuration บางอย่างไว้สำหรับการทดสอบแบบคร่าวๆ โดยไม่ต้องแก้ไขไฟล์ config.yml ไปมา

7.1.2 run_face.py, run_palm.py และ run_walk.py

ไฟล์ทั้งสามใช้สำหรับสั่งรันเฉพาะข้อมูลชีวมิติประเภทเดียว โดยรับ argument ในรูปแบบ ที่ยืดหยุ่น 3 แบบ ดังนี้

ตัวอย่างการใช้งาน

แบบที่ 1: ระบุโฟลเดอร์และรหัสอาสาสมัคร

```
python run_palm.py data 002
```

แบบที่ 2: ระบุโฟลเดอร์ของอาสาสมัครโดยตรง

```
python run_palm.py data/002
```

แบบที่ 3: ระบุไฟล์ที่ต้องการตรวจสอบโดยตรง

```
python run_palm.py data/002/002_palm_L_N.jpg
```

ทั้งสามแบบระบบจะค้นหาไฟล์ในโฟลเดอร์แบบลึกลงไปไม่เกิน 5 ระดับชั้น เช่นเดียวกับ run_folder.py

option ที่ใช้ร่วมกันทั้งสามไฟล์ (run_face.py, run_palm.py และ run_walk.py)

Option	ความหมาย
--config PATH	ไฟล์ตั้งค่าที่ต้องการใช้ (ค่าเริ่มต้น config.yml)
--id ID	ระบุรหัสอาสาสมัครเอง แทนการอ่านจากชื่อไฟล์
--out-root DIR	โฟลเดอร์สำหรับเขียนผลลัพธ์
--no-overlay	ไม่สร้างไฟล์ overlay
--quiet	ไม่แสดงผลการเช็คแต่ละรายการระหว่างรัน

option เฉพาะของ run_face.py

Option	ความหมาย
--sample-fps F	จำนวนเฟรมที่สุ่มต่อวินาที (ค่าเริ่มต้น 5)
--overlay PATH	ระบุที่อยู่ของไฟล์ overlay เอง
--fail-fast	หยุดประมวลผลทันทีเมื่อพบข้อบกพร่องระดับไฟล์ เช่น ไฟล์เปิดอ่านไม่ได้

option เฉพาะของ run_palm.py

Option	ความหมาย
--no-detect	ตรวจสอบเฉพาะ metadata โดยไม่เรียกใช้โมเดลตรวจจับมือ
--no-angle-3d-tabs	ไม่สร้างไฟล์ HTML แสดงมุมฝ่ามือแบบสามมิติ

ไฟล์ HTML แสดงมุมฝ่ามือแบบสามมิติ (palm_<id>_angle3d_tabs.html) ถูกสร้างขึ้นโดยค่าเริ่มต้น เป็นไฟล์สำหรับตรวจสอบด้วยสายตาว่ามุมที่ระบบวัดได้ตรงกับท่าที่อาสาสมัครทำจริงหรือไม่ โดยแสดงตำแหน่ง landmark ของมือในปริภูมิสามมิติที่หมุนดูได้

ส่วน --no-detect มีประโยชน์เมื่อต้องการตรวจสอบเพียงว่าไฟล์ภาพมีนามสกุลและความละเอียดถูกต้องหรือไม่ซึ่งรันได้เร็วกว่ามากเนื่องจากไม่ต้องโหลดและเรียกใช้โมเดล

option เฉพาะของ run_walk.py

Option	ความหมาย
--sample-fps F	จำนวนเฟรมที่สุ่มต่อวินาที (ค่าเริ่มต้น 5.0)
--fail-fast	หยุดประมวลผลทันทีเมื่อพบข้อบกพร่องเชิงโครงสร้าง
--no-progress	ไม่แสดงผลการเช็คแต่ละรายการระหว่างรัน

7.2 การใช้งานผ่านการเรียก API

ผู้ใช้งานสามารถใช้งานผ่านการเรียก API ได้ โดยสามารถอ่านขั้นตอนและรายการ endpoint ได้ในหัวข้อที่ 8.

Docker และ API โดยสามารถอ่านขั้นตอนการใช้งานอย่างง่ายได้ในหัวข้อ 8.3 ลำดับการใช้งานทั่วไป

กรณีต้องการใช้ API โดยตรง สามารถรันคำสั่งเพื่อเริ่มต้น server ได้

```
uvicorn main:app --reload --port 8002
```

ตัวเลือก `--reload` ใช้สำหรับการพัฒนาระบบเท่านั้น เนื่องจาก server จะเริ่มการทำงานใหม่ทุกครั้งที่ไฟล์โปรแกรมเปลี่ยนแปลง สำหรับการใช้งานจริงผ่าน Docker จะเริ่มด้วยคำสั่ง `uvicorn main:app --host 0.0.0.0 --port 8000` ตามที่กำหนดใน Dockerfile

หลังจากนั้นเปิดหน้า <http://localhost:8002/docs> เพื่อเข้าใช้หน้า biometric qc api ซึ่งเป็น interactive UI ให้สามารถลองเรียกใช้ endpoint ได้จากหน้าเว็บโดยตรง

7.3 การใช้งาน Streamlit dashboard (api_tester.py)

เป็นช่องทางที่ใช้งานง่ายที่สุด เหมาะกับผู้ใช้ที่ไม่ต้องการเขียนคำสั่ง command line หรือเรียกใช้ API เอง โดยเป็นหน้าเว็บที่ผู้ใช้สามารถอัปโหลดไฟล์ ติดตามความคืบหน้า และดาวน์โหลดผลลัพธ์ได้ ในหน้าเดียว

ทั้งนี้ Streamlit dashboard นี้ทำงานโดยการเรียกใช้ API เบื้องหลัง ไม่ได้เรียกใช้ pipeline โดยตรง ดังนั้น จึงต้องเปิดใช้งาน API ก่อนเสมอ เป็นเหมือนส่วนที่มีไว้เพื่อทดสอบ API

ขั้นตอนการเปิดใช้งาน

ขั้นตอนที่ 1 : เปิดใช้งาน API ก่อน โดยเลือกวิธีใดวิธีหนึ่ง

วิธีที่ 1: รันผ่าน Docker (แนะนำการรันด้วยวิธีนี้)

```
docker compose up -d --build
```

วิธีที่ 2: รันโดยตรงบนเครื่อง

```
uvicorn main:app --reload
```

ขั้นตอนที่ 2 : เปิด Streamlit dashboard ในหน้าต่าง terminal อีกหน้าต่างหนึ่ง

```
streamlit run api_tester.py
```

จะแสดงผลลัพธ์

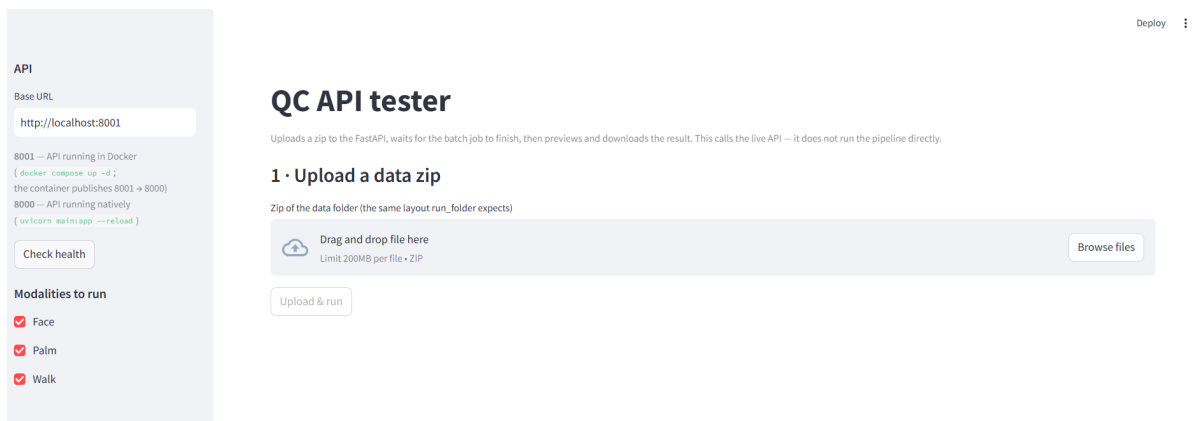
You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>

Network URL: ... (port 8501)

หลังจากนั้นเบราว์เซอร์จะเปิดหน้า dashboard ขึ้นมาโดยอัตโนมัติ

ตัวอย่างของหน้า dashboard



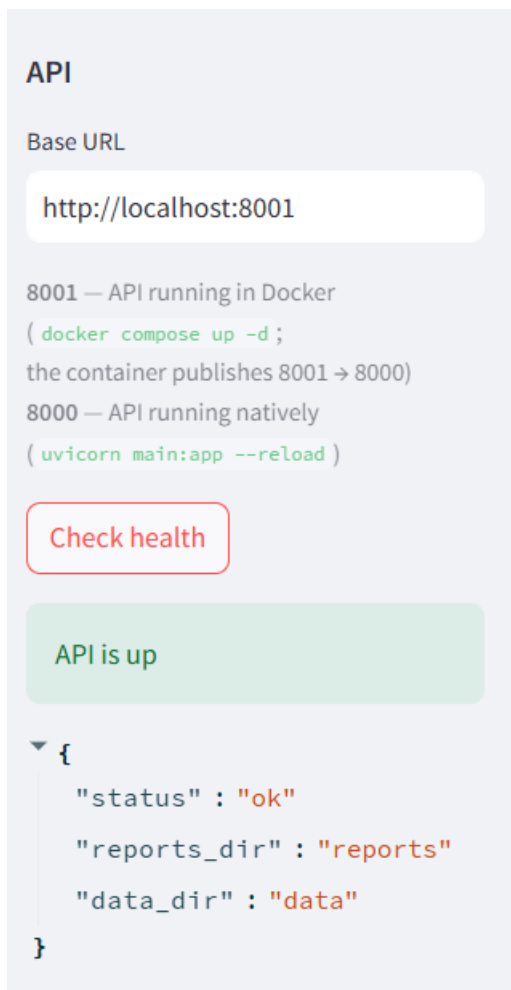
ส่วนประกอบของหน้าจอ

หน้า dashboard แบ่งออกเป็นแถบด้านข้าง (sidebar) สำหรับตั้งค่าและพื้นที่หลักที่แบ่ง เป็น 4 ขั้นตอนตามลำดับการใช้งาน

ส่วน	หน้าที่
Base URL	ที่อยู่ของ API ที่ต้องการเชื่อมต่อ
ปุ่ม Check health	ทดสอบว่าเชื่อมต่อกับ API ได้หรือไม่
Modalities to run	เลือกประเภทข้อมูลชีวมิติที่ต้องการตรวจสอบ (Face, Palm, Walk)

ช่อง Base URL สามารถแก้ไขได้ ทำให้สามารถเชื่อมต่อกับ API ที่ติดตั้งอยู่บนเครื่องอื่นได้ โดยค่าเริ่มต้นคือ `http://localhost:8000` ซึ่งเป็นค่าสำหรับกรณีที่รัน API โดยตรงบนเครื่อง หากรัน API ผ่าน Docker ต้องแก้ไขค่านี้เป็น `http://localhost:8001` ตามที่ได้อธิบาย เรื่องหมายเลข port ไว้ในหัวข้อ 8.1

ก่อนเริ่มใช้งานทุกครั้ง ควรกดปุ่ม Check health เพื่อยืนยันว่าเชื่อมต่อกับ API ได้ก่อน ซึ่งจะแสดงโพลเดอร์ข้อมูลและโพลเดอร์ผลลัพธ์ที่ API กำลังใช้งานอยู่ด้วย



ขั้นตอนที่ 1 : Upload a data zip

อัปโหลดไฟล์ .zip ที่บรรจุข้อมูลชีวิตของอาสาสมัคร โดยรองรับไฟล์ .zip เพียงไฟล์เดียว ต่อการรันหนึ่งครั้ง จากนั้นกดปุ่ม Upload & run เพื่อเริ่มการตรวจสอบ โครงสร้างภายในไฟล์ .zip สามารถเป็นแบบใดก็ได้ เนื่องจากระบบจะค้นหาไฟล์ในโฟลเดอร์ระดับความลึกลงไปไม่เกิน 5 ระดับชั้น

ขั้นตอนที่ 2 : Job status

แสดงสถานะและความคืบหน้าของงานตรวจสอบ โดยมีปุ่ม Poll until complete สำหรับสั่งให้ระบบตรวจสอบสถานะซ้ำโดยอัตโนมัติจนกว่างานจะเสร็จสิ้น เนื่องจากการตรวจสอบข้อมูลจำนวนมากใช้เวลานาน ระบบจึงออกแบบให้ทำงานเบื้องหลังและให้ผู้ใช้ติดตามความคืบหน้าแทนการรอการตอบกลับ โดยระบบจะตรวจสอบสถานะซ้ำ ทุก 2 วินาที และรอได้นานที่สุดประมาณ 200 นาที

ขั้นตอนที่ 3 : Results

แสดงผลลัพธ์การตรวจสอบในรูปแบบตาราง โดยแสดงเฉพาะรายการที่มีสถานะ FAIL เป็นหลัก

ขั้นตอนที่ 4 : Download

ดาวน์โหลดไฟล์ผลลัพธ์ของการรันครั้งนั้น

ข้อจำกัดของการใช้งานโดยวิธีนี้ คือ Streamlit dashboard ปรับแต่ง option ได้น้อยที่สุดในบรรดาทั้งสามวิธี โดยสามารถเลือกได้เพียงประเภทข้อมูลชีวิตที่ต้องการตรวจสอบเท่านั้น ส่วน option อื่นๆ เช่น --sample-fps, --no-overlay, หรือ --fail-fast จะใช้ค่าเริ่มต้น default ของ API ทั้งหมด หากต้องการปรับแต่งค่าเหล่านี้ต้องใช้ช่องทาง command line หรือเรียกใช้ API โดยตรง นอกจากนี้ยังรองรับการอัปโหลดเฉพาะไฟล์ .zip เท่านั้น ในขณะที่การเรียกใช้ API โดยตรง รองรับการอัปโหลดไฟล์หลายไฟล์พร้อมกันได้

ตัวอย่างผลลัพธ์การรันจนครบทุกขั้นตอนมีดังนี้

API

Base URL

http://localhost:8001

8001 — API running in Docker

(docker compose up -d ;

the container publishes 8001 → 8000)

8000 — API running natively

(uvicorn main:app --reload)

Check health

Modalities to run

☒ Face
 ☒ Palm
 ☒ Walk

☒ Face
 ☒ Palm
 ☒ Walk

☒ Face
 ☒ Palm
 ☒ Walk

Deploy

QC API tester

Uploads a zip to the FastAPI, waits for the batch job to finish, then previews and downloads the result. This calls the live API — It does not run the pipeline directly.

1 • Upload a data zip

Zip of the data folder (the same layout run_folder expects)

Drag and drop file here

Limit 200MB per file • ZIP

Browse files

test_check_batch.zip 42.5MB

×

Upload & run

2 • Job status

job_id: c8b283341a85

Status: completed

Job completed.

3 • Results

10 total rows • 4 FAIL rows

FAIL rows

volunteer_id	data_type	filename	overall_status	check_name	reason
001	face_rgb	001_face_rgb.mp4	FAIL	check_duration	duration=25.8 < 40
001	face_rgb	001_face_rgb.mp4	FAIL	check_face_size	frame=0 face=170x209 < 180x180 [judged fail-ratio=100% (m
001	face_rgb	001_face_rgb.mp4	FAIL	check_turn_down	down detected but hold too short: 1.8s < 2.0s
001	face_rgb	001_face_rgb.mp4	FAIL	check_turn_front	front hold check failed in matched 'audio_prompt' window:

Preview all rows (PASS + FAIL)

4 • Download

What to download ?

☐ Summary bundle (all_summary + filenames)
 ☐ all_summary only
 ☐ filenames only (validate_filenames)
 ☒ Whole reports folder

☒ Include overlay media (large)

Prepare download

นอกจากนี้โปรแกรมมีไฟล์ Streamlit dashboard อีกหนึ่งไฟล์คือ view_results.py ซึ่งมีวัตถุประสงค์ต่างกัน โดยสั้นเชิง view_results.py ไม่ได้เชื่อมต่อกับ API แต่อ่านไฟล์ผลลัพธ์จากโฟลเดอร์โดยตรง ใช้สำหรับ ตรวจสอบผลลัพธ์รายอาสาสมัครโดยละเอียด และเปรียบเทียบผลของอาสาสมัครสองรายได้ รายละเอียดของ ไฟล์นี้อยู่ในภาคผนวก หัวข้อ 9.

8. Docker และ API

ระบบตรวจสอบข้อมูลชีวมิตินี้สามารถติดตั้งและใช้งานผ่าน Docker container เพื่อให้สามารถนำไปติดตั้งบนเครื่องใดก็ได้โดยไม่ต้องติดตั้งไลบรารีต่างๆ ด้วยตนเอง และเปิดให้เรียกใช้งานผ่าน API ที่พัฒนาด้วย FastAPI

8.1 การติดตั้งและรัน Docker

โครงสร้างไฟล์ที่เกี่ยวข้อง (ทุกไฟล์อยู่ใน root directory)

ไฟล์	หน้าที่
Dockerfile	กำหนดขั้นตอนการสร้าง image กำหนด python version, base OS, ไลบรารี, และติดตั้ง dependencies ต่างๆ
docker-compose.yml	กำหนดค่าการรัน container เช่น การตั้งหมายเลข port และการ mount โฟลเดอร์ เพื่อให้ใช้คำสั่งรันได้ง่ายโดยไม่ต้องกำหนด option จำนวนมากใน command line
.dockerignore	กำหนดว่าไฟล์ใดจะไม่ถูกคัดลอกเข้าไปใน image เพื่อให้ image ขนาดไม่ใหญ่จนเกินไป
requirements.txt	รายชื่อ Python libraries ที่จะติดตั้งพร้อมเลขเวอร์ชัน

ขั้นตอนการติดตั้งและรัน Docker

1. ติดตั้ง Docker Desktop บนเครื่องที่จะใช้งาน และเปิด Docker Desktop
2. เตรียมไฟล์โมเดล โมเดลทั้ง 4 ไฟล์ไม่ได้ถูกบรรจุไว้ใน image เนื่องจากมีขนาดใหญ่ จึงต้องดาวน์โหลดมาไว้ในโฟลเดอร์ models/ บนเครื่องก่อน แล้วจึง mount เข้าไปในขั้นตอนการรัน ได้แก่

models/

```
face_landmarker.task    # สำหรับตรวจสอบใบหน้า
hand_landmarker.task    # สำหรับตรวจสอบฝ่ามือ
pose_landmarker.task    # สำหรับตรวจสอบท่าทางการเดิน
yolov8n.pt              # สำหรับตรวจสอบสิ่งกีดขวางในวิดีโอท่าทางการเดิน
```

3. เตรียมโฟลเดอร์ data/ และ reports/ สำหรับข้อมูลนำเข้าและผลลัพธ์

การรันด้วย Docker Compose

วิธีนี้เป็นวิธีที่แนะนำมากที่สุดเนื่องจากการตั้งค่าทั้งหมด ได้แก่ ชื่อ image, การ map port, และการ mount โฟลเดอร์ ถูกกำหนดไว้ในไฟล์ docker-compose.yml แล้ว

คำสั่ง	ความหมาย
docker compose up -d --build	สร้าง image (หากยังไม่มีหรือมีการแก้ไขโค้ด) และรัน container ใน background
docker compose logs -f	ดู logs ของ container แบบต่อเนื่อง
docker compose down	หยุดและลบ container

เมื่อรันสำเร็จแล้ว สามารถเปิดหน้าเอกสาร API ได้ที่ <http://localhost:8001/docs>

สรุปหน้าที่เรียกเปิดได้หลังรัน container

URL	ระบบที่เปิด	การใช้งาน
http://localhost:8001/docs	FastAPI Swagger UI	เรียกและทดสอบ API เช่น ตรวจสอบสถานะระบบ อัปโหลดข้อมูล ตรวจสอบความคืบหน้าและดาวน์โหลดผลลัพธ์
http://localhost:8501	api_tester.py dashboard	อัปโหลดไฟล์และเรียกใช้งาน API ผ่านหน้าเว็บ โดยไม่ต้องเรียก endpoint ด้วยตนเอง
http://localhost:8502	view_results.py dashboard	เปิดดูและวิเคราะห์ไฟล์ผลลัพธ์ที่อยู่ในโฟลเดอร์ reports/

หากต้องการใช้งานจากคอมพิวเตอร์เครื่องอื่นในเครือข่ายเดียวกัน ให้แทน localhost ด้วย IP address ของเครื่องที่รัน Docker Compose เช่น <http://192.168.1.50:8501>

การรันด้วยคำสั่ง docker โดยตรง

ในกรณีที่ไม่ต้องการใช้ docker compose สามารถใช้คำสั่งดังนี้

```
# bash
```

```
# สร้าง image
```

```
docker build -t biometric-qc .
```

รัน container พร้อม mount โฟลเดอร์ที่จำเป็น

```
docker run -d --name biometric-qc-api \
```

```
-p 8001:8000 \
```

```
-v ./models:/app/models \
```

```
-v ./data:/app/data \
```

```
-v ./reports:/app/reports \
```

```
biometric-qc
```

หมายเลข port ที่ใช้มี 2 ตัว ซึ่งมีความหมายคือ port 8000 คือ port ที่โปรแกรมทำงานอยู่ภายใน container ซึ่งกำหนดไว้ใน Dockerfile ส่วน port 8001 คือ port ที่เปิดให้เข้าถึงจากภายนอก container ซึ่งกำหนดไว้ใน docker-compose.yml ในรูปแบบ 8001:8000 ดังนั้นผู้ใช้งานต้องเข้าถึงระบบผ่าน port 8001 ตามที่กำหนดไว้ใน docker-compose.yml หากต้องการเปลี่ยนเป็น port อื่น สามารถแก้ไขตัวเลขด้านซ้ายในไฟล์ docker-compose.yml ได้ เช่น เปลี่ยนเป็น 8080:8000 แล้วเข้าถึงผ่าน <http://localhost:8080/docs>

การ mount โฟลเดอร์

โฟลเดอร์ 3 โฟลเดอร์ถูก mount จากเครื่องเข้าไปใน container ดังนี้

โฟลเดอร์บนเครื่อง	container	หน้าที่
./models	/app/models	ไฟล์โมเดล (อ่าน)
./data	/app/data	ข้อมูลชีวมิตินำเข้า (อ่าน)
./reports	/app/reports	ผลลัพธ์การตรวจสอบ (เขียนกลับไปที่เครื่อง)

การ mount โฟลเดอร์ reports/ มีความสำคัญทำให้ผลลัพธ์ที่โปรแกรมเขียนออกมาถูกบันทึกกลับไปที่เครื่องของผู้ใช้ หากไม่ mount ผลลัพธ์จะอยู่ใน container เท่านั้นและหายไปเมื่อลบ container เหตุผลที่ข้อมูลชีวมิติและไฟล์โมเดลไม่ถูกบรรจุไว้ใน image มีสองประการ ประการแรกคือขนาด เพราะทั้งข้อมูลวิดีโอและไฟล์โมเดลมีขนาดใหญ่มาก การบรรจุไว้ใน image จะทำให้ image มีขนาดใหญ่เกินความจำเป็น ประการที่สองคือความเป็นส่วนตัวของข้อมูล ข้อมูลชีวมิติของอาสาสมัครเป็น sensitive data จึงไม่ควรใส่ใน image

การตั้งค่าโมเดล YOLO

ใน Dockerfile มีการกำหนดตัวแปร environment สำหรับไลบรารี Ultralytics (ซึ่งเป็นไลบรารีของโมเดล YOLO) ดังนี้

```
ENV YOLO_CONFIG_DIR=/app/.ultralytics \
```

```
YOLO_OFFLINE=1 \
```

```
ULTRALYTICS_ANALYTICS=False
```

YOLO_CONFIG_DIR กำหนดให้ไฟล์ตั้งค่าของ YOLO ถูกเก็บไว้ในโฟลเดอร์โครงการ แทนที่จะเก็บไว้ในโฟลเดอร์ home ของผู้ใช้ เพื่อให้ container สามารถทำงานได้เหมือนกันทุกเครื่อง, YOLO_OFFLINE=1 ป้องกันมิให้ไลบรารีพยายามดาวน์โหลดไฟล์โมเดลจากอินเทอร์เน็ตขณะทำงานเนื่องจากไฟล์โมเดลถูก mount เข้ามาแล้ว, ULTRALYTICS_ANALYTICS=False ปิดการส่งข้อมูลการใช้งานกลับไปยังผู้พัฒนาไลบรารี

8.2 การเรียกใช้งานผ่าน API

เมื่อรัน container สำเร็จแล้ว สามารถเรียกใช้งาน API ได้ที่ <http://localhost:8001> โดย FastAPI จะสร้างหน้าเอกสารประกอบการใช้งานแบบ interactive ไว้แล้วที่ <http://localhost:8001/docs> ซึ่งผู้ใช้สามารถทดลองเรียกใช้ endpoint ต่างๆ ได้จากหน้าเว็บโดยตรงโดยไม่ต้องเขียนโปรแกรมเรียกใช้

ตารางสรุป endpoint

Method	Endpoint	หน้าที่
GET	/health	ดูสถานะว่าระบบทำงานอยู่หรือไม่ และเช็คตำแหน่ง directory ข้อมูลนำเข้า และข้อมูลส่งออก
POST	/checks/batch	ส่งตรวจสอบข้อมูลทั้งหมดในโฟลเดอร์ data/ ในเครื่อง local
GET	/checks/batch	ดูรายการงานตรวจสอบทั้งหมด
GET	/checks/batch/{job_id}	ดูความคืบหน้าของ jobs ในการตรวจสอบ
POST	/checks/uploads	อัปโหลดข้อมูล (zip หรือหลายๆ ไฟล์) และส่งรันตรวจสอบ
POST	/checks/file	อัปโหลดไฟล์เดียวเพื่อตรวจสอบ
GET	/results	อ่านผลลัพธ์รวมทุกอาสาสมัคร
GET	/results/download	ดาวน์โหลดไฟล์ผลลัพธ์ เลือกได้ว่าจะมีเพียงไฟล์สรุป หรือมีไฟล์ผลลัพธ์รายบุคคลของแต่ละอาสาสมัครด้วย
GET	/results/{volunteer_id}	อ่านผลลัพธ์ของอาสาสมัครรายหนึ่ง

GET	/volunteers	ดูรายชื่ออาสาสมัครที่มีผลลัพธ์แล้ว
-----	-------------	------------------------------------

8.2.1 GET /health

ใช้ตรวจสอบว่าระบบทำงานอยู่หรือไม่ และดูว่าระบบกำลังอ่านข้อมูลจากโฟลเดอร์ใด

ตัวอย่างการใช้งาน

bash

```
curl http://localhost:8001/health
```

ตัวอย่างผลลัพธ์ที่ได้

json

```
{
  "status": "ok",
  "reports_dir": "reports",
  "data_dir": "data"
}
```

8.2.2 POST /checks/uploads

เป็น endpoint หลักที่ใช้ในการตรวจสอบข้อมูล โดยผู้ใช้อัปโหลดชุดข้อมูลชีวมิติเข้ามาแล้วระบบจะตรวจสอบให้ทั้งหมด ซึ่งรองรับการอัปโหลด 2 รูปแบบ ได้แก่

- ไฟล์ .zip หนึ่งไฟล์ ระบบจะแตกไฟล์ให้อัตโนมัติ
- ไฟล์หลายไฟล์ จากการเลือกทั้งโฟลเดอร์ ระบบจะรักษาโครงสร้างโฟลเดอร์ไว้

ตัวอย่างการใช้งาน

bash

```
curl -X POST "http://localhost:8001/checks/uploads" \
  -F "files=@data.zip"
```

สามารถระบุ option เพิ่มเติมได้ผ่าน query parameter ดังนี้

Parameter	default	ความหมาย
run_face	true	ตรวจสอบข้อมูลใบหน้าหรือไม่
run_palm	true	ตรวจสอบข้อมูลฝ่ามือหรือไม่
run_walk	true	ตรวจสอบข้อมูลท่าทางการเดินหรือไม่
sample_fps	1.0	จำนวนเฟรมที่สุ่มต่อวินาทีของวิดีโอต้นฉบับ หากใส่ 0 หรือค่าติดลบจะประมวลผลทุกเฟรม

ตัวอย่างการตรวจสอบเฉพาะข้อมูลฝ่ามือ

bash

```
curl -X POST "http://localhost:8001/checks/uploads?run_face=false&run_walk=false" \
-F "files=@data.zip"
```

ผลลัพธ์ที่คาดว่าจะได้คือรหัสงาน (job_id) พร้อมสถานะ HTTP 202 Accepted ซึ่งหมายความว่าระบบรับงานไว้แล้วและกำลังประมวลผลอยู่เบื้องหลัง ซึ่งผู้ใช้นำ job_id ไปตรวจสอบความคืบหน้าต่อไป การอัปโหลดแต่ละครั้งจะถูกแตกไฟล์ไว้ในโฟลเดอร์ชั่วคราว และเขียนผลลัพธ์ลงในโฟลเดอร์ผลลัพธ์เฉพาะของการอัปโหลดนั้น จึงไม่ปะปนกับข้อมูลในโฟลเดอร์ data/ ในเครื่อง local ที่ใช้ร่วมกัน และไม่ปะปนกับผลลัพธ์ของการอัปโหลดครั้งอื่น

8.2.3 POST /checks/batch

ทำหน้าที่เหมือน /checks/uploads แต่ตรวจสอบข้อมูลที่มีอยู่แล้วในโฟลเดอร์ data/ ของเครื่อง local server โดยไม่ต้องอัปโหลด เหมาะกับกรณีที่ผู้ใช้ mount โฟลเดอร์ข้อมูลเข้ามาที่ container โดยตรงแล้ว

ตัวอย่างการใช้งาน

bash

```
curl -X POST "http://localhost:8001/checks/batch"
```

รองรับ query parameter ชุดเดียวกันกับ /checks/uploads ทุกประการ และคืนค่ากลับเป็น job_id พร้อมสถานะ 202 Accepted เช่นเดียวกัน

8.2.4 GET /checks/batch/{job_id} และ GET /checks/batch

GET /checks/batch/{job_id} ใช้ตรวจสอบความคืบหน้าของงานที่สั่งไว้ โดยนำ job_id ที่ได้จากขั้นตอนก่อนหน้ามาใส่

ตัวอย่างการใช้งาน

bash

```
curl http://localhost:8001/checks/batch/{job_id}
```

ผู้ใช้งานเรียก endpoint นี้ซ้ำหากมีสถานะเป็น running แสดงว่ายังรันงานนี้อยู่ยังไม่เสร็จ และเรียกเป็นระยะจนกว่าสถานะจะเปลี่ยนเป็น completed จึงจะสามารถอ่านผลลัพธ์ได้ เนื่องจากการตรวจสอบข้อมูลจำนวนมากใช้เวลานาน ระบบจึงออกแบบให้ทำงานเบื้องหลังและไม่ให้ผู้ใช้งานต้องรอการตอบกลับเป็นเวลานาน นอกจากนี้ยังสามารถดูรายการงานทั้งหมดได้ที่ GET /checks/batch และกรองตามสถานะได้ด้วย query parameter status เช่น running, completed, หรือ failed

8.2.5 POST /checks/file

ใช้ตรวจสอบไฟล์เดียวและได้ผลลัพธ์กลับมาทันที โดยไม่ต้องรอแบบ batch เหมาะกับการทดสอบไฟล์เดียว

ตัวอย่างการใช้งาน

bash

```
curl -X POST "http://localhost:8001/checks/file" \  
-F "file=@001_face_rgb.mp4"
```

ระบบจะอ่านชื่อไฟล์เพื่อตัดสินใจว่าเป็นข้อมูลชีวมิติประเภทใด แล้วจึงส่งไปยัง pipeline ที่เหมาะสม ผลลัพธ์ที่เป็นไปได้มี 3 กรณี ดังนี้

ประเภทไฟล์	สถานะ HTTP	ผลลัพธ์
face_rgb, walk_F, walk_S	200	ตรวจสอบและคืนผลลัพธ์ได้ทันที
palm_* และไฟล์ใบหน้าประเภทอื่น	501	ไม่รองรับการตรวจสอบไฟล์เดียว
ชื่อไฟล์ไม่ตรงกับข้อกำหนดใด	422	ปฏิเสธไฟล์

เหตุผลที่ข้อมูลฝ่ามือไม่รองรับการตรวจสอบไฟล์เดี่ยว เนื่องจากการตรวจสอบฝ่ามือมีการเช็คบางรายการที่อาจต้องพิจารณาที่ระดับอาสาศัมครรวมทุก 10 ไฟล์ และไฟล์ใบหน้าอื่นๆ เช่นไฟล์จากเครื่องถ่ายภาพความร้อน ระบบไม่ได้อรองรับการตรวจสอบไฟล์เหล่านี้ ทำให้ระบบคืนสถานะ 501 สำหรับผู้ใช้ที่ต้องการตรวจสอบข้อมูลฝ่ามือควรใช้ /checks/batch หรือ /checks/uploads แทน

8.2.6 GET /results

อ่านผลลัพธ์รวมของทุกอาสาศัมคร ในรูปแบบรายการผลการเช็คแต่ละรายการ

bash

อ่านผลลัพธ์ทั้งหมด (ทั้ง PASS และ FAIL)

curl "http://localhost:8001/results"

อ่านเฉพาะรายการที่ไม่ผ่าน

curl "http://localhost:8001/results?status=FAIL"

อ่านผลลัพธ์ของการอัปโหลดครั้งหนึ่งโดยเฉพาะ

curl "http://localhost:8001/results?job_id={job_id}"

หากไม่ระบุ job_id ระบบจะอ่านผลลัพธ์จากโฟลเดอร์ reports/ ที่ใช้ร่วมกัน แต่หากระบุ job_id ระบบจะอ่านผลลัพธ์เฉพาะของการอัปโหลดครั้งนั้น

8.2.7 GET /results/download

ดาวน์โหลดไฟล์ผลลัพธ์ โดยเลือกขอบเขตของไฟล์ที่ต้องการได้ผ่าน parameter scope

scope	ผลลัพธ์ที่ได้
summary (ค่า default)	ไฟล์ .zip บรรจุ all_summary และ filenames
full	ไฟล์ .zip บรรจุทั้งโฟลเดอร์ผลลัพธ์ รวมโฟลเดอร์ย่อยรายอาสาศัมคร
all_summary	ไฟล์ all_summary เพียงไฟล์เดียว
filenames	ไฟล์ filenames เพียงไฟล์เดียว

ตัวอย่างการเรียกใช้งาน

bash

ค่าเริ่มต้น: ได้ทั้ง all_summary และ filenames ในไฟล์ zip เดียว

```
curl -o reports_summary.zip "http://localhost:8001/results/download"
```

เลือกรูปแบบไฟล์เป็น JSON

```
curl -o reports_summary.zip "http://localhost:8001/results/download?format=json"
```

ดาวน์โหลดทั้งไฟล์เดอร์ผลลัพธ์

```
curl -o reports_full.zip "http://localhost:8001/results/download?scope=full"
```

ดาวน์โหลดทั้งไฟล์เดอร์ พร้อมไฟล์ overlay สำหรับตรวจสอบด้วยสายตา

```
curl -o reports_full.zip \  
    "http://localhost:8001/results/download?scope=full&include_overlays=true"
```

ดาวน์โหลดเฉพาะไฟล์ใดไฟล์หนึ่ง

```
curl -o all_summary.csv "http://localhost:8001/results/download?scope=all_summary"
```

```
curl -o filenames.csv "http://localhost:8001/results/download?scope=filenames"
```

ดาวน์โหลดผลลัพธ์ของการอัปเดตครั้งหนึ่งโดยเฉพาะ

```
curl -o reports_summary.zip \  
    "http://localhost:8001/results/download?job_id={job_id}"
```

ค่า default เป็นไฟล์ .zip ที่บรรจุสองไฟล์ โดย all_summary ตอบว่าไฟล์ที่อาสาสมัครส่งมาผ่านข้อกำหนดหรือไม่ ส่วน filenames ตอบว่าอาสาสมัครส่งไฟล์มาครบถ้วนและตั้งชื่อถูกต้องหรือไม่ ซึ่งผู้ใช้งานจำเป็นต้องใช้ทั้งสองไฟล์ประกอบกันจึงจะสรุปผลได้ครบถ้วนตามข้อกำหนด

ในกรณีที่ไฟล์ใดไฟล์หนึ่งขาดหายไป เช่น สั่งรันโดยข้ามขั้นตอนการตรวจสอบชื่อไฟล์ด้วย option --no-filenames ระบบจะยังคงส่งไฟล์ .zip ที่บรรจุเฉพาะไฟล์ที่มีอยู่กลับมา และ จะแจ้งสถานะ 404 ก็ต่อเมื่อไม่พบไฟล์ทั้งสองไฟล์เท่านั้น

parameter include_overlays (true / false)

ใช้ได้กับ scope=full เท่านั้น โดยค่าเริ่มต้นเป็น false ซึ่งจะไม่รวมไฟล์ overlay เนื่องจากไฟล์ overlay เป็นไฟล์สำหรับตรวจสอบด้วยสายตา ซึ่งจะมีขนาดใหญ่กว่าไฟล์ผลลัพธ์อื่นๆ มีข้อควรระวังเรื่องขนาดไฟล์ การใช้ scope=full ร่วมกับ include_overlays=true กับข้อมูลจำนวนมากจะทำให้ได้ไฟล์ขนาดใหญ่มากในการตอบกลับเพียงครั้งเดียว ซึ่งอาจทำให้การเชื่อมต่อหมดเวลาก่อนดาวน์โหลดเสร็จ ในกรณีที่ต้องการไฟล์ overlay ของข้อมูลชุดใหญ่ แนะนำให้สั่งรันผ่าน command line แล้วอ่านจากโฟลเดอร์ reports/ โดยตรงแทน

parameter format (csv / json)

ใช้เลือกระหว่าง csv (ค่าเริ่มต้น) และ json โดยมีผลกับ scope=summary, scope=all_summary, และ scope=filenames แต่ไม่มีผลกับ scope=full ซึ่งจะส่ง ไฟล์ทุกไฟล์ตามที่มีอยู่ในโฟลเดอร์

โครงสร้างภายในไฟล์ .zip

กรณี scope=summary ไฟล์ .zip จะบรรจุไฟล์ 2 ไฟล์ที่ระดับบนสุด

reports.zip

```
|—— all_summary.csv  
|—— filenames.csv
```

กรณี scope=full ไฟล์ .zip จะรักษาโครงสร้างโฟลเดอร์เดิมไว้ทั้งหมด ทำให้เมื่อแตกไฟล์ ออกมาแล้วจะได้

โครงสร้างเหมือนโฟลเดอร์ reports/ ที่ได้จากการสั่งรันผ่าน command line

reports.zip

```
|—— all_summary.csv  
|—— all_summary.json
```

```

|----- filenames.csv
|----- filenames.json
|----- face_summary.csv
|----- palm_summary.csv
|----- walk_summary.csv
|----- 001/
|   |----- face_001_overall.csv
|   |----- face_001_result.csv
|   |----- ...
|----- 002/
|   |----- ...

```

8.2.8 GET /results/{volunteer_id} และ GET /volunteers

ใช้อ่านผลลัพธ์ของอาสาสมัครรายหนึ่งโดยเฉพาะ และดูรายชื่ออาสาสมัครที่มีผลลัพธ์แล้ว

```
bash
```

```
# ดูรายชื่ออาสาสมัครที่มีผลลัพธ์
```

```
curl "http://localhost:8001/volunteers"
```

```
# อ่านผลลัพธ์ของอาสาสมัครหมายเลข 001
```

```
curl "http://localhost:8001/results/001"
```

ผลลัพธ์ของ /results/{volunteer_id} ประกอบด้วยผลสรุปรวม (overall) และรายการผลการเช็คแต่ละรายการ (checks) ของอาสาสมัครรายนั้น หากไม่พบผลลัพธ์ของอาสาสมัครรายนั้นจะคืนสถานะ 404

8.3 ลำดับการใช้งานทั่วไป

8.3.1 ลำดับการใช้งานทั่วไปบนเครื่องเดียว

ลำดับการใช้งานระบบผ่าน API อย่างง่ายมีดังนี้

1. เปิด Docker Desktop รัน container ด้วยคำสั่ง `docker compose up -d --build`
2. เปิด `http://localhost:8001/docs` หากต้องการใช้งาน FastAPI Swagger UI

URL	ระบบที่เปิด	การใช้งาน
<code>http://localhost:8001/docs</code>	FastAPI Swagger UI	เรียกและทดสอบ API เช่น ตรวจสอบสถานะระบบ อัปโหลดข้อมูล ตรวจสอบความคืบหน้าและดาวน์โหลดผลลัพธ์
<code>http://localhost:8501</code>	<code>api_tester.py</code> dashboard	อัปโหลดไฟล์และเรียกใช้งาน API ผ่านหน้าเว็บโดยไม่ต้องเรียก endpoint ด้วยตนเอง
<code>http://localhost:8502</code>	<code>view_results.py</code> dashboard	เปิดดูและวิเคราะห์ไฟล์ผลลัพธ์ที่อยู่ในโฟลเดอร์ <code>reports/</code>

3. ตรวจสอบว่าระบบพร้อมทำงาน ด้วย `GET /health`
 4. อัปโหลดข้อมูลและสั่งตรวจสอบ ด้วย `POST /checks/uploads` แล้วบันทึก `job_id` ที่ได้
 5. ตรวจสอบความคืบหน้า ด้วย `GET /checks/batch/{job_id}` ซ้ำเป็นระยะจนสถานะเป็น `completed`
 6. อ่านผลลัพธ์ ด้วย `GET /results?job_id={job_id}` หรือดาวน์โหลดไฟล์ผลลัพธ์ ด้วย `GET /results/download?job_id={job_id}` ซึ่งจะได้ทั้ง `all_summary` และ `filenames` ในไฟล์ `.zip`
- ทั้งนี้ ผู้ใช้ที่ไม่ต้องการเรียกใช้ API ด้วยตนเอง สามารถใช้งานผ่าน Streamlit dashboard (`api_tester.py`) ซึ่งเรียกใช้ API ชุดเดียวกันนี้เบื้องหลัง ตามที่ได้กล่าวไว้ในหัวข้อ 7.3

8.3.2 ลำดับการใช้งานทั่วไปบนหลายเครื่องในเครือข่ายเดียวกัน

ระบบสามารถจำลองการเรียกใช้งาน API จากคอมพิวเตอร์เครื่องอื่นได้ โดยให้คอมพิวเตอร์เครื่องหนึ่งทำหน้าที่เป็นเครื่องเซิร์ฟเวอร์สำหรับรัน Docker Compose และให้คอมพิวเตอร์อีกเครื่องทำหน้าที่เป็นผู้ใช้งานระบบ ทั้งสองเครื่องต้องเชื่อมต่ออยู่ในเครือข่ายเดียวกัน เช่น Wi-Fi หรือ LAN เดียวกัน

1. การเตรียมเครื่องเซิร์ฟเวอร์

บนคอมพิวเตอร์เครื่องที่ใช้รันระบบ ให้เปิด Docker Desktop และรันคำสั่งต่อไปนี้จากโฟลเดอร์โปรเจกต์

```
docker compose up -d --build
```

ตรวจสอบสถานะของ container ด้วยคำสั่ง

```
docker compose ps
```

จากนั้นค้นหา IPv4 address ของเครื่องเซิร์ฟเวอร์ด้วยคำสั่ง

```
ipconfig
```

ตัวอย่าง IPv4 address ที่ได้คือ

192.168.1.50

2. การเปิดระบบจากคอมพิวเตอร์เครื่องอื่น

บนคอมพิวเตอร์อีกเครื่องหนึ่ง ให้แทน localhost ด้วย IPv4 address ของเครื่องเซิร์ฟเวอร์ ดังตัวอย่างต่อไปนี้

URL ที่เปิดจากคอมพิวเตอร์เครื่องอื่น	ระบบที่เปิด	การใช้งาน
http://192.168.1.50:8001/docs	FastAPI Swagger UI	ทดสอบการเรียก API โดยตรง เช่น ตรวจสอบสถานะระบบ อัปโหลดข้อมูล ตรวจสอบความคืบหน้า และดาวน์โหลดผลลัพธ์
http://192.168.1.50:8501	api_tester.py dashboard	อัปโหลดไฟล์และเรียกใช้งาน API ผ่านหน้าเว็บโดยไม่ต้องเรียก endpoint ด้วยตนเอง
http://192.168.1.50:8502	view_results.py dashboard	เปิดดูและวิเคราะห์ผลลัพธ์ที่ระบบบันทึกไว้ในโฟลเดอร์ reports/

ในตัวอย่างนี้ 192.168.1.50 คือ IPv4 address ของเครื่องที่รัน Docker Compose ผู้ใช้ต้องเปลี่ยนค่านี้ให้ตรงกับ IP address ของเครื่องเซิร์ฟเวอร์

3. การจำลองการเรียก API จากอีกเครื่องหนึ่ง

สามารถเปิด

http://<server-ip>:8001/docs

จากคอมพิวเตอร์เครื่องอื่น แล้วดำเนินการตามลำดับดังนี้

- เรียก GET /health เพื่อตรวจสอบว่าสามารถเชื่อมต่อกับ API ได้
- เรียก POST /checks/uploads เพื่ออัปโหลดไฟล์ ZIP และเริ่มการตรวจสอบ
- บันทึกค่า job_id ที่ได้รับจากระบบ
- เรียก GET /checks/batch/{job_id} เป็นระยะเพื่อตรวจสอบสถานะและความคืบหน้า
- เมื่อสถานะเป็น completed ให้เรียก GET /results?job_id={job_id} เพื่ออ่านผลลัพธ์
- เรียก GET /results/download?job_id={job_id} เพื่อดาวน์โหลดรายงาน

วิธีนี้เป็นการจำลองการใช้งานจริงเนื่องจากคำขอถูกส่งจากคอมพิวเตอร์ผู้ใช้ผ่านเครือข่ายไปยัง FastAPI ที่ทำงานอยู่บนอีกเครื่องหนึ่ง

4. การใช้งาน api_tester.py dashboard จากอีกเครื่องหนึ่ง

ผู้ใช้สามารถเปิด dashboard ได้จาก

http://<server-ip>:8501

จากนั้นอัปโหลดไฟล์ ZIP เลือกประเภทข้อมูลชีวมิติที่ต้องการตรวจสอบ และติดตามสถานะของงานผ่านหน้าเว็บได้ เมื่อ api_tester.py และ FastAPI ทำงานอยู่ภายใน Docker Compose เดียวกัน ค่า API Base URL ภายใน dashboard ควรเป็น

http://biometric-qc-api:8000

ค่านี้เป็นชื่อ service และ port ภายในเครือข่ายของ Docker ไม่ใช่ URL ที่ผู้ใช้เปิดใน browser

ความสัมพันธ์ของแต่ละ URL มีดังนี้

คอมพิวเตอร์ผู้ใช้

→ เปิด **http://<server-ip>:8501**

→ api_tester.py container

→ เรียก **http://biometric-qc-api:8000**

→ FastAPI container

ดังนั้น แม้ว่าผู้ใช้จะเปิด dashboard ผ่าน port 8501 จากคอมพิวเตอร์เครื่องอื่น แต่ dashboard จะเรียก API ผ่าน port 8000 ภายใน Docker โดยอัตโนมัติ ผู้ใช้ไม่จำเป็นต้องเปลี่ยน API Base URL เป็น IP address ของเครื่องตนเอง

ในกรณีที่รัน api_tester.py โดยตรงบนคอมพิวเตอร์ผู้ใช้และไม่ได้รันผ่าน Docker Compose ค่า API Base URL ต้องเปลี่ยนเป็น

http://<server-ip>:8001

5. การตั้งค่าเครือข่าย

หากคอมพิวเตอร์เครื่องอื่นไม่สามารถเปิด URL ได้ ให้ตรวจสอบดังนี้

- คอมพิวเตอร์ทั้งสองเครื่องเชื่อมต่ออยู่ในเครือข่ายเดียวกัน
- เครื่องเซิร์ฟเวอร์ใช้ network profile แบบ Private
- Windows Firewall อนุญาตการเชื่อมต่อผ่าน TCP port 8001, 8501 และ 8502
- Docker Desktop และ container ทั้งสามรายการกำลังทำงาน
- IP address ของเครื่องเซิร์ฟเวอร์ยังไม่เปลี่ยนแปลง

การใช้งานภายในเครือข่ายเดียวกันไม่จำเป็นต้องตั้งค่า port forwarding บนเราเตอร์

ระบบเวอร์ชันปัจจุบันยังไม่มีระบบยืนยันตัวตนจึงควรเปิดให้ใช้งานเฉพาะภายในเครือข่ายที่เชื่อถือได้ และไม่ควรเปิด port เหล่านี้ให้เข้าถึงได้โดยตรงจากอินเทอร์เน็ตสาธารณะ

9. ภาคผนวก

9.1 การใช้งาน view_results.py dashboard

ไฟล์ : `view_results.py`

นอกเหนือจาก Streamlit dashboard สำหรับสั่งรันการตรวจสอบ (`api_tester.py`) ในหัวข้อ 7.3 ระบบยังมี dashboard อีกตัวหนึ่งสำหรับตรวจสอบผลลัพธ์ที่ได้จากการรันไปแล้ว ความแตกต่างของทั้งสองคือ `api_tester.py` ทำหน้าที่ส่งข้อมูลเข้าระบบและสั่งรันผ่าน API ในขณะที่ `view_results.py` ไม่เชื่อมต่อกับ API และไม่สั่งรันการตรวจสอบใดๆ แต่อ่านไฟล์ผลลัพธ์ในโฟลเดอร์ `reports/` ที่อยู่ใน local server มาแสดงผลเท่านั้น ซึ่งเหมาะกับการตรวจสอบผลลัพธ์หลังการรันเสร็จสิ้น

การเรียกใช้งาน

`pip install streamlit pandas altair`

`streamlit run view_results.py`

โดยหากต้องการอ่านผลลัพธ์จากโฟลเดอร์อื่นที่ไม่ใช่ `reports/` ให้ระบุด้วยตัวเลือก `--reports` โดยต้องมีเครื่องหมาย `--` คั่น ก่อนตัวเลือกเสมอ เครื่องหมาย `--` มีความจำเป็นเนื่องจาก Streamlit ต้องการแยก
ระหว่างตัวเลือกของ Streamlit เองกับตัวเลือกของโปรแกรม หากไม่ใส่ Streamlit จะตีความว่า `--reports` เป็นตัวเลือกของตนเองและแจ้งข้อผิดพลาด ตัวอย่างการรันเช่น

`streamlit run view_results.py -- --reports reports/test_batch`

ซึ่งหากไม่พบโฟลเดอร์ที่ระบุ dashboard จะแจ้งเตือนพร้อมแนะนำให้รัน `python run_folder.py data` ก่อน

ไฟล์ที่ dashboard อ่าน

dashboard อ่านไฟล์ผลลัพธ์ที่ `run_folder.py` สร้างไว้ โดยไม่ประมวลผลเพื่อเช็คข้อมูลซ้ำมิติใหม่

ระดับ	ไฟล์	ใช้ในโหมด
reports/	all_summary.csv / .json	All failures
reports/	face_summary.csv, palm_summary.csv, walk_summary.csv	Summaries

reports/	filenames.csv	Single
reports/<id>/	*_overall.csv	Single, Compare
reports/<id>/	*_result.csv	Single, Compare
reports/<id>/	*_detail.csv	Single
reports/<id>/	*_detail_header.csv	Single (กราฟมุมการหันศีรษะ)

โหมดการใช้งาน 4 โหมด

เลือกโหมดได้จากแถบด้านซ้าย

1. **Single** ตรวจสอบอาสาสมัครรายเดียว จะแสดงผลของอาสาสมัคร 1 รายอย่างละเอียด ประกอบด้วย

- ผลการตรวจสอบชื่อไฟล์ว่าส่งครบ 17 ไฟล์หรือไม่
- ผลสรุปแยกตามชีวมิติทั้งสามประเภท
- ตารางผลการเช็ครายการ พร้อมเหตุผลของแต่ละรายการ
- กราฟมุมการหันศีรษะตามเวลา แสดงค่า yaw, pitch, roll ของทุกเฟรม พร้อมเส้น threshold ทำให้เห็นได้ว่าอาสาสมัครหันศีรษะถึงมุมที่กำหนดหรือไม่ และค้างทำไว้นานเพียงใด
- ข้อมูลรายละเอียดในแต่ละเฟรม

2. **Compare** เพื่อเปรียบเทียบอาสาสมัคร 2 ราย จะเลือกอาสาสมัคร 2 รายเพื่อเปรียบเทียบผลการเช็คแต่ละรายการในตารางเดียวกัน ทำให้เห็นความแตกต่างได้ชัดเจน เหมาะกับกรณีที่ต้องการหาสาเหตุว่าเหตุใดอาสาสมัครรายหนึ่งไม่ผ่านการตรวจสอบในขณะที่อีกรายผ่าน

3. **Summaries** ผลสรุปแยกตามชีวมิติ อ่านไฟล์สรุปที่รากของโฟลเดอร์ reports/ โดยเลือกได้ว่าจะดูชีวมิติประเภทใด แสดงผล 1 แถวต่อ 1 การเช็คที่ไม่ผ่าน พร้อมตัวเลือกให้แสดงเฉพาะรายการที่ FAIL

4. **All failures** ผลรวมทุกชีวมิติ อ่าน all_summary.csv และ all_summary.json ซึ่งเป็นผลสรุปรวมข้ามชีวมิติ แสดงเฉพาะรายการที่ FAIL พร้อมเหตุผล และสามารถดาวน์โหลดเป็นไฟล์ CSV หรือ JSON ได้ โหมดนี้เหมาะกับการใช้งานเป็นลำดับแรกหลังการรันเสร็จสิ้น เพื่อดูภาพรวมว่ามีปัญหาที่ใดบ้าง ก่อนเจาะลงไปดูรายละเอียดของอาสาสมัครรายที่มีปัญหาในโหมด Single

การแสดงผลสถานะด้วยสี

dashboard กำหนดสีประจำแต่ละสถานะเพื่อให้อ่านผลได้เร็วขึ้น

สถานะ	ความหมาย
PASS / COMPLETE	ผ่านเกณฑ์ / ส่งไฟล์ครบ
FAIL / INCOMPLETE / UNRECOGNISED	ไม่ผ่านเกณฑ์ / ส่งไฟล์ไม่ครบ / ชื่อไฟล์ไม่ตรงข้อกำหนด
SKIP	ข้ามการเช็คในเฟรมนั้นๆ ไป
ERROR	เกิดข้อผิดพลาดระหว่างการประมวลผล
MISSING	ไม่มีผลการตรวจสอบสำหรับรายการนั้น

ลำดับการใช้งานที่แนะนำ

1. รันการตรวจสอบด้วย `python run_folder.py data`
2. เปิด dashboard ด้วย Streamlit run `view_results.py`
3. เริ่มจากโหมด All failures เพื่อดูภาพรวมว่ามีปัญหาที่ใดบ้าง
4. ดูรายละเอียดไปที่โหมด Single สำหรับอาสาสมัครที่มีปัญหา เพื่อดูเหตุผลรายการเช็คและกราฟมุมการหันศีรษะ
5. หากต้องการเทียบกับอาสาสมัครที่ผ่านการตรวจสอบ ใช้โหมด Compare